

KERNEL BORDERLANDS

Out-of-Band Attestation Node

OAN

A Hardware-Rooted Trust Anchor for Kernel Borderlands, Explained From First Principles

Design Concept Document • v1.0 • August 2026

What this document is

A complete, illustrated walkthrough of OAN: why Kernel Borderlands needs a piece of hardware that lives outside the machine it protects, how that hardware is put together, how it talks to the host and to itself, how it scales from one desk to a lab, and what it costs to build. Written so a reader with no prior security-hardware background can follow every diagram end to end.

KERNEL BORDERLANDS TEAM

Karthik • PardhuVarma • Tejaswini • Rupa

<https://github.com/pardhuvarmax/kernel-borderlands>

Contents

1	What Is OAN?	4
2	The Problem OAN Solves	4
2.1	Software Watchdogs Have a Ceiling	4
2.2	Moving the Last Line of Defense Off the Host	5
3	A Worked Example: Anatomy of a Full Kernel Compromise	5
3.1	The Scenario	5
3.2	Minute Zero: What the Attacker Can Reach	5
3.3	Where OAN Picks Up	6
4	Design Philosophy	7
5	High-Level Architecture	7
6	Two Communication Planes	8
7	Meet the Three Processors	9
7.1	SBC — The Brain	9
7.2	STM32 — The Spinal Reflex	10
7.2.1	Why a Dedicated Chip, Not the SBC or the ESP32	10
7.3	ESP32 — The Mouth	10
8	TPM 2.0 and the Relay: Trust and Fencing	11
8.1	TPM 2.0 — Hardware Root of Trust	11
8.2	Relay Controller — Physical Fencing	11
8.3	Display	11
9	What OAN Does, Continuously	12
10	The Attestation Heartbeat	12
11	Where OAN Sits in the Trust Boundary	13
12	The Recovery Ladder	13
13	Scaling to a Fleet: the Shared Chassis Design	14
13.1	What Is Shared, and What Every Host Still Needs	14
13.2	Why Cap at Five Hosts	15
14	Fabric Management: One Node to a Fleet	15
14.1	The Local Autonomy Principle	16
14.2	What Crosses the Fabric Plane	16
14.3	Fleet Responsibilities	17
14.4	Policy Inheritance	17
15	Deployment Walkthrough: From One Desk to a Lab	18
16	Building One: Bill of Materials	18

16.1 Shared Chassis Parts (bought once per chassis)	19
16.2 Per-Host Channel Parts	19
16.3 Total Cost by Fleet Size	19
17 Building It on a Budget	20
18 The Production Appliance	21
19 Why This Architecture Works	21
20 Security Properties and Honest Limits	22
20.1 What OAN Guarantees	22
20.2 What OAN Does Not Do	22
21 What-if: Boot/Rootkits?	23
21.1 The Question, Stated Precisely	23
21.2 Already Closed: Self-Containment	24
21.2.1 Hardening CPM's Own Data Structures	24
21.3 Getting the Key Out of Host Memory	25
21.4 The Critical Piece: Sealing Key Usage to Measured Boot State	26
21.4.1 The Mechanism	26
21.4.2 Before and After, From the Attacker's Side	28
21.4.3 Demo Environment: Trying This on Real Hardware	28
21.5 Narrowing the Memory-Corruption Gap	30
21.5.1 Preventive Hardening, Already Mostly Free	30
21.5.2 Confirmed on kb-core's Own Userspace Daemon	32
21.5.3 Closing the Bypass in the Measured-Boot Chain Itself	32
21.5.4 What Survives All of It: a Same-Privilege-Level Last Resort	32
21.6 Cross-Checking Views That Should Never Disagree	33
21.7 Timing, Not Just Content	34
21.8 Cross-Checking the Clock Itself	35
21.9 Classify the Failure, Don't Just Fence	35
21.10 Letting Weak Signals Accumulate Across the Fleet	36
21.11 What This Closes, and What Remains Open	37
22 How OAN Compares to Existing Approaches	38
23 Frequently Asked Questions	38
24 Positioning: What OAN Adds to Kernel Borderlands	39
25 From Design to Deployment: The Build Roadmap	39
26 Glossary	40
27 Build Checklist	42
27.1 Before Ordering Parts	42
27.2 Assembling the Shared Chassis	42
27.3 Wiring the First Host	42
27.4 Scaling the Chassis	42

27.5 Hardening Against Boot-Time and Rootkit Compromise (Optional, §21)	43
28 Summary Cheat-Sheet	43
29 Appendix: Document Map	44
30 The Complete Picture: Full System Diagram	45
31 References	47

1 What Is OAN?

Kernel Borderlands (KB) [1] protects a Linux machine from the inside: an eBPF sensor watches the kernel at Ring 0, a Go control plane scores what it sees, and a Rust watchdog called `kb-checker` keeps an eye on the rest of the stack from a separate process on the same box. That covers almost every attack. It does not cover one: what happens if the attacker gets *all the way in* — full control of the kernel itself, the same privilege level the sensor runs at.

The **Out-of-Band Attestation Node (OAN)** is Kernel Borderlands’ answer to that one remaining case. It is a small, physically separate computer — its own processor, its own memory, its own power supply, its own wiring — that sits next to the protected machine and asks it one question, over and over, many times a second: “*are you still telling the truth?*” If the answer stops making sense, OAN can cut the machine’s power itself, without needing any cooperation from the machine it is cutting off.

Where OAN stands today. OAN is a design, not yet a shipped product — there is no code, no wire protocol, and no dedicated repository directory for it yet. This document is the architecture that KB intends to build: everything described below is the design target, laid out at the level of detail needed to start building it. Where a number is a cost estimate rather than a fixed price, it is presented as a range for exactly that reason.

In Plain Terms

Think of Kernel Borderlands today as a very well-guarded building, with guards standing *inside* the building watching every door and hallway. OAN is a second guard station across the street, watching the building from outside, with its own phone line, its own lights, and its own switch for the building’s power — so that even if every guard inside the building is somehow tricked or overpowered, the guard across the street still notices, and can still act.

The rest of this document builds that picture up piece by piece: why the problem exists (§2), the four design rules OAN follows (§4), how its hardware is laid out (§5–§8), how it actually watches the host (§10–§12), how one unit grows into a fleet (§13–§14), and finally what it costs to build one (§16–§17).

2 The Problem OAN Solves

2.1 Software Watchdogs Have a Ceiling

`kb-checker` already exists to solve a version of this problem. Its entire job is to watch the rest of Kernel Borderlands from outside their failure domain: it runs as its own process, it keeps zero persistent state, it opens zero network sockets, and it leans on operating-system primitives it trusts more than it trusts KB’s own code. That design is deliberately simple (KISS: Keep It Simple, Stupid) so the watchdog itself stays small enough to actually verify by reading it.

But “outside their failure domain” only stretches so far when everything is still running on the *same* machine. `kb-checker` is a separate *process*, not a separate *computer*. It still shares:

- the same kernel, which arbitrates every socket it talks over,

- the same scheduler, which decides when (or whether) it gets to run,
- the same power and reset lines, which a kernel-level attacker ultimately also controls.

If an attacker achieves a full kernel compromise, they are operating at the same privilege level as the eBPF sensor that is supposed to catch them, and above the privilege level of the watchdog that is supposed to notice. Software-only isolation on a single host has a ceiling: the watchdog and the thing it watches share one root of trust — the CPU and the kernel running on it.

Analogy

Imagine a company’s internal auditor, who reports to the same CEO as everyone they audit. Even a very honest, very independent-minded auditor is still, structurally, inside the same organization. If the CEO themselves is the problem, the auditor’s independence has a limit built into the org chart. OAN is what happens when you hire an *external* auditor who reports to nobody inside the building at all.

2.2 Moving the Last Line of Defense Off the Host

OAN’s design goal is simple to state: move the very last line of defense *off the host entirely*. A physically separate node, with its own compute and its own power domain, keeps attesting to the host’s integrity and keeps the ability to fence it, without depending on that host’s kernel being honest about anything. OAN does not replace **kb-checker** — it gives Kernel Borderlands an external anchor point for the one failure mode no same-host component can structurally cover alone: total kernel compromise.

This mirrors a decision Kernel Borderlands already made once, at a different layer. The Critical Process Module (CPM) exists because “the security platform’s own components must be immune to the failure modes they’re meant to prevent” at the *authorization* layer — OAN applies that same principle one layer further down, at the *hardware* layer.

3 A Worked Example: Anatomy of a Full Kernel Compromise

Abstract descriptions of “kernel compromise” are easy to nod along to and hard to actually picture. This section walks through one concrete timeline, end to end, showing exactly where a software-only stack runs out of road, and exactly where OAN picks up.

3.1 The Scenario

An attacker has found and exploited a memory-safety bug in a third-party kernel module loaded on the protected host. The exploit gives them arbitrary code execution at Ring 0 — the same privilege level **kb-core**’s own eBPF sensors run at. From this point on, the attacker is not “a process being watched by the kernel”; they *are* the kernel, in every way that matters to software running on top of it.

3.2 Minute Zero: What the Attacker Can Reach

Once Ring 0 is theirs, the attacker can, in principle, do all of the following without any of it looking unusual from inside the host:

- Patch the eBPF sensor’s own view of the syscall table, so **kb-core** keeps reporting a clean

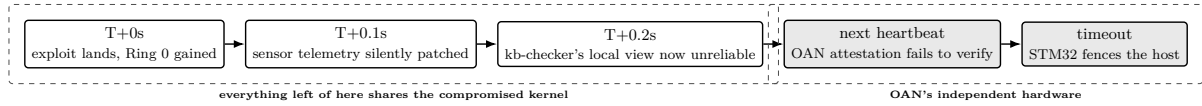


Figure 1: The same timeline, seen from two different trust domains. The left three boxes all run on the compromised kernel; the right two never do.

process tree.

- Starve or suspend the **kb-checker** process at the scheduler level, so it never gets CPU time to notice anything is wrong.
- Forge normal-looking heartbeats on any socket the kernel itself arbitrates — because the kernel arbitrates all of them.
- Leave the host’s power and reset lines exactly as they were, so nothing about the machine’s outward behavior changes.

In Plain Terms

This is the precise moment every *purely software* defense runs out of road. Every one of the bullets above is a case of “ask the kernel whether the kernel is honest” — and a fully compromised kernel will always answer yes. There is no cleverer question you can ask it in software that gets around this, because software questions all have to travel through the same compromised kernel to get an answer.

3.3 Where OAN Picks Up

None of the four bullets above touch OAN, for a simple structural reason: OAN never asked the kernel a question that the kernel could answer on the attacker’s behalf in the first place.

1. **The next heartbeat is due.** **kb-checker**’s attestation either does not arrive on the OOB link at all (because the process was starved), or arrives but fails TPM verification (because the sealed measurements no longer match).
2. **The SBC does not reset the STM32’s liveness timer.** It only ever does this after a valid, verified attestation — and there is not one.
3. **The STM32’s own clock, running independently of anything on the host or the SBC, reaches the configured timeout.**
4. **The fail-closed policy fires.** The relay — wired directly to the STM32, never to the SBC or the host — cuts power or reset, physically, regardless of what the compromised kernel is doing or reporting at that instant.

The attacker’s Ring 0 access, which was total with respect to the host, grants them precisely nothing with respect to the STM32’s timer or the relay it drives — those never took input from the host to begin with. That gap is the entire reason OAN exists.

4 Design Philosophy

Everything about OAN follows from four rules. They deliberately mirror **kb-checker**’s own KISS pillars, moved from the software layer to the hardware layer.

Principle	What it means in practice
Out-of-Band	OAN never depends on the monitored operating system for its own execution. It has its own boot chain, its own clock, its own storage.
Independent	Own CPU, own memory, own storage, own power. No shared failure domain with the host — if the host’s power supply dies, loses network, or is compromised, none of that touches OAN.
Hardware-Rooted	Trust originates from a physical chip (a TPM 2.0 module), not from a software claim that could itself be forged by a compromised kernel.
Fail-Safe (configurable)	OAN can take protective action even when the host cannot cooperate. The exact failure behavior — fence immediately, or leave the host running and only alert — is a deployment-level policy switch, described in §10, defaulting to the safer of the two.

Read together, these four rules answer the same question from four different angles: *“what happens to OAN’s trustworthiness if the host is fully owned by an attacker?”* The answer, by design, is nothing — OAN keeps working exactly as before, because nothing about its own operation ever routed through the host in the first place.

5 High-Level Architecture

At the top level, OAN sits beside the existing Kernel Borderlands host and talks to exactly one of its five subsystems directly: **kb-checker**. It never talks to **kb-core** or **kb-control-plane** directly, because **kb-checker** is already the one KB component whose entire purpose is external verification — it is the natural point of contact on the host side.

Three things in Figure 2 carry the whole design:

- **The OOB link is not the host’s network.** It is a dedicated serial line or a dedicated Ethernet port that never touches the host’s normal network stack, so a compromised host cannot reach it the way it could reach a regular NIC.
- **The relay is driven by the STM32, not the SBC.** Fencing survives a full Linux crash on OAN’s own SBC, because the chip that controls the relay never runs Linux at all (§7.2).
- **Fleet traffic is a separate, dashed line.** Anything that talks to other chassis goes out through the ESP32 on a completely different physical channel (§6).

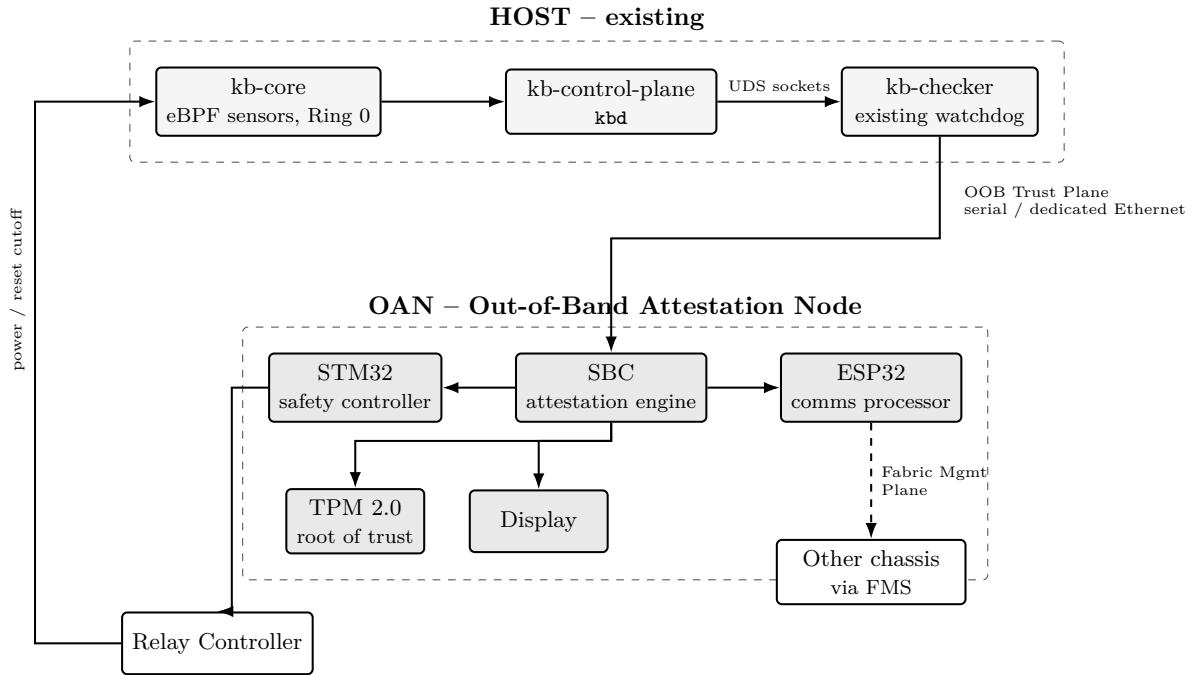


Figure 2: OAN sits beside the host, talking only to **kb-checker** over a dedicated out-of-band link. The relay gives OAN a physical, software-independent way to cut the host’s power.

6 Two Communication Planes

OAN exposes two communication channels that never share a wire, a switch, or a protocol stack. Keeping them apart is what lets OAN scale to a fleet (§13) without ever weakening the one link that actually matters for trust.

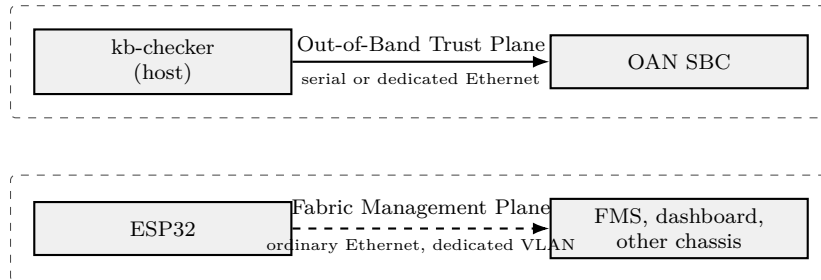


Figure 3: Two planes, two purposes. The top plane establishes trust; the bottom plane only coordinates a fleet that already trusts itself locally.

	OOB Trust Plane	Fabric Management Plane
Carries	Attestation, heartbeat, recovery/fencing triggers	Fleet discovery, policy sync, dashboards, event aggregation
Owner chip	SBC (direct link)	ESP32
Shared between hosts?	Never — physical, point-to-point per host	Yes — ordinary networking across the fleet
Required for basic protection?	Yes — this <i>is</i> the trust boundary	No — see the Local Autonomy Principle, §14.1

In Plain Terms

The Fabric Management Plane is allowed to be “just Ethernet” precisely because it never carries anything security-critical. It only ever sees signed summaries that OAN’s own trust plane has already vouched for (§14) — so even a fully compromised management network can’t forge a clean bill of health for a host that is actually compromised.

7 Meet the Three Processors

OAN’s brain is deliberately split across three separate chips, each with exactly one job. The split looks like extra hardware at first glance, but it is the single most important decision in the whole design: it means no one chip is ever both “the thing that makes the security decision” and “the thing that runs a full general-purpose, network-facing operating system.”

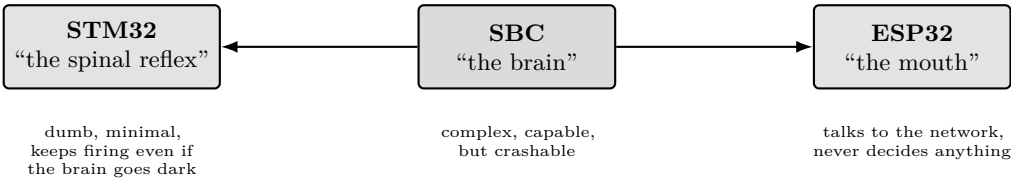


Figure 4: The three-chip split, in one picture. Each chip is deliberately worse at the other two chips’ jobs than it is at its own.

7.1 SBC — The Brain

The SBC (a small board like a Raspberry Pi, or an Intel N100 mini PC for the shared-chassis design, §13) is the counterpart to `kb-checker`’s role, but running on independent hardware. It carries:

- the main attestation engine and policy engine,
- cluster management (the Fabric Management Service, §14),
- the local dashboard and database,
- cryptographic verification of everything the host reports,
- logging.

This is the only chip in OAN complex enough to run a full Linux userspace, and therefore the only one complex enough to have bugs, hang, or crash. Everything downstream of it is built assuming exactly that will eventually happen.

7.2 STM32 — The Spinal Reflex

The STM32 is a tiny microcontroller with one job: watch a timer, and flip a relay if the timer runs out. It has no operating system, no network stack, and no scheduler — just one small, deterministic firmware loop.

- Hardware independent watchdog timer (IWDG).
- Heartbeat supervision, on its own clock.
- Relay control and emergency fencing.
- Safe-state controller for the whole appliance.

Analogy

A spinal reflex — pulling your hand off a hot stove — happens before the signal even reaches your brain. It is fast precisely because it is *not* doing any complicated thinking. The STM32 is built the same way: it is not smart enough to be tricked, because it is not smart enough to do anything except watch a clock and flip a switch.

7.2.1 Why a Dedicated Chip, Not the SBC or the ESP32

Two questions come up immediately, and both have concrete answers.

Why not let the SBC drive the relay directly? Because then fencing depends on Linux being alive and scheduled promptly. If the SBC hangs, panics, or is simply busy at the wrong moment, the fencing decision hangs with it — recreating, one level down, the exact same-failure-domain problem OAN exists to solve for the host in the first place.

Why not use the ESP32, since it is already on the board? Because the ESP32 runs a full Wi-Fi and networking stack — a much larger, more remotely-reachable attack surface, and the chip most likely to be running stock SDK code with known vulnerabilities. Handing fencing authority to the least-trusted, most network-exposed chip on the board would be exactly backwards.

The STM32 costs a few dollars precisely *because* it does not do policy or attestation logic — that stays on the SBC. It only watches a timer and flips a relay, a job small enough for a commodity board to do with very high confidence, no custom safety-rated silicon required.

7.3 ESP32 — The Mouth

The ESP32 is the dedicated management and communications processor. It handles device discovery, secure provisioning, the maintenance interface, and optional wireless management — and it is the chip that owns the Fabric Management Plane (§6).

The ESP32 never makes a security decision on its own. It only carries traffic. That single restriction is what makes it safe for the ESP32 to also be the chip most exposed to ordinary networking: a compromise of the ESP32 or the management plane it carries cannot reach the

attestation or relay path at all, because that path lives entirely on the SBC/STM32/TPM side of the board.

8 TPM 2.0 and the Relay: Trust and Fencing

8.1 TPM 2.0 — Hardware Root of Trust

A Trusted Platform Module (TPM) is a small, dedicated security chip built for exactly one purpose: storing cryptographic keys and integrity measurements in a way that even the computer it is attached to cannot tamper with from software alone. OAN's TPM does double duty:

- **Seals OAN's own integrity state.** Measured boot on the SBC and STM32 firmware is sealed against the TPM, so OAN can prove its own software has not been tampered with — the same property it is asking the host to prove about itself.
- **Verifies kb-checker's reported state.** Each attestation `kb-checker` sends over the OOB link is checked against sealed reference measurements in the TPM, so a compromised host kernel cannot simply forge a clean report.

In Plain Terms

Sealing *both* sides against the same physical chip closes a gap that would otherwise exist: it is not enough for OAN to trust the host — OAN's own word has to be worth something too. A TPM that only checked the host, sitting inside an OAN whose own integrity nobody verified, would just move the same trust problem one hop over rather than solving it.

8.2 Relay Controller — Physical Fencing

The relay is the final, physical link in the chain: a switch that can cut or restore the host's power or reset line, wired directly to the STM32 (§7.2), never to the SBC. It is used only after software-level recovery has already failed (§12), and its behavior on link loss is a configured policy rather than a fixed reflex:

Policy	Behavior
Fail-closed (default)	If the OOB link drops, or an attestation is missing or invalid past the configured timeout, the STM32 fences the host — favoring protection over uptime.
Fail-open	The host is left running and an alert is raised instead; chosen deliberately for availability-critical hosts where an unattended, unverified fencing event would itself be an unacceptable outage.

Fail-closed is the default because it matches the same principle the Critical Process Module already applies elsewhere in Kernel Borderlands: fail closed toward protection. A deployment can opt a specific host into fail-open, but it has to do so explicitly.

8.3 Display

A small display (an I2C OLED or LCD panel) shows host status, integrity state, threat level, heartbeat, cluster health, TPM status, relay status, and a rolling event log. It is a read-only

reflection of state the SBC and STM32 already hold — useful for a technician standing in front of the chassis, but never an input to any decision OAN makes.

9 What OAN Does, Continuously

Once running, OAN’s loop never stops:

- Verifies host integrity against sealed TPM measurements.
- Confirms **kb-checker** liveness on every heartbeat.
- Validates each attestation the host sends.
- Detects heartbeat loss and suspicious silence.
- Logs every security-relevant event locally.
- Coordinates software-level recovery before ever reaching for the relay.
- Performs last-resort hardware fencing when every other option is exhausted.

10 The Attestation Heartbeat

Every interval, **kb-checker** on the host sends a signed attestation across the OOB link. The SBC checks it against the TPM, and — if it is valid — resets a liveness timer on the STM32. That timer is the entire mechanism: as long as valid attestations keep arriving on schedule, the STM32 has nothing to do. The moment they stop, or start failing verification, the STM32’s own clock takes over.

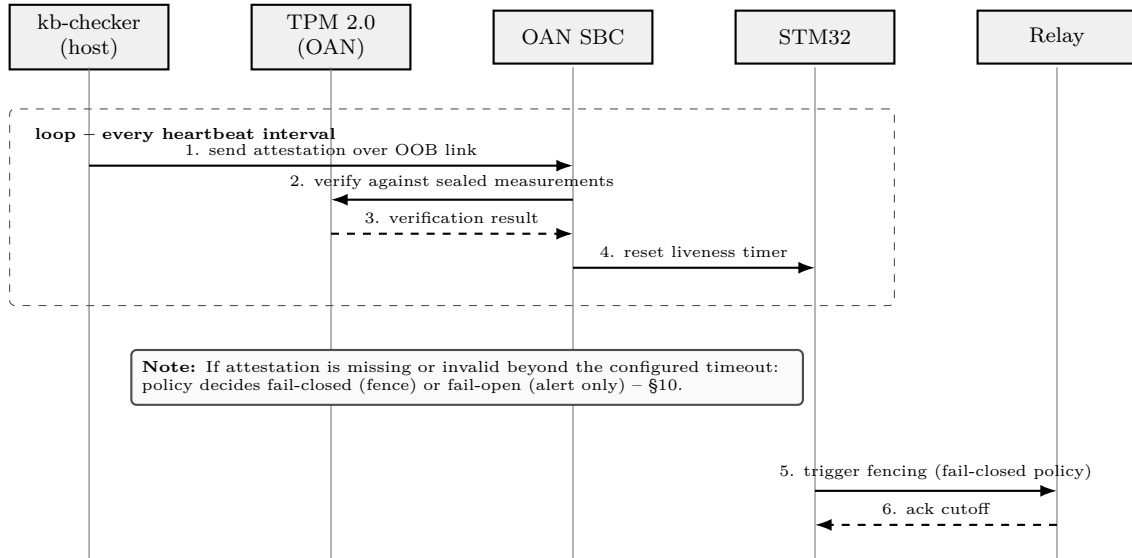


Figure 5: The attestation heartbeat. Steps 1–4 repeat forever while everything is healthy; steps 5–6 only fire when the loop breaks and the fail-closed policy is active.

In Plain Terms

This is why the STM32 does not need to understand cryptography at all. It never verifies an attestation itself — it only watches whether the SBC *keeps telling it to reset the clock*. Verification happens once, on the SBC, where the TPM lives; the STM32's only job downstream of that is noticing silence.

11 Where OAN Sits in the Trust Boundary

Zooming out, OAN's place in the overall Kernel Borderlands data flow is a single, sharp line:

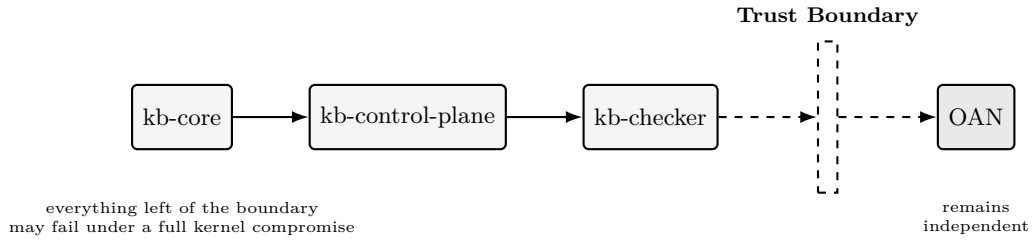


Figure 6: Everything to the left of the trust boundary shares one kernel, and so shares one failure mode. OAN is the only component structurally guaranteed to still be telling the truth on the right.

12 The Recovery Ladder

Hardware fencing is the *last* resort, not the first. It sits at the bottom of a five-stage recovery ladder, built on top of **kb-checker**'s existing three-stage software chain (`systemctl stop` → `SIGKILL` → `iptables drop`). OAN owns only the final stage.

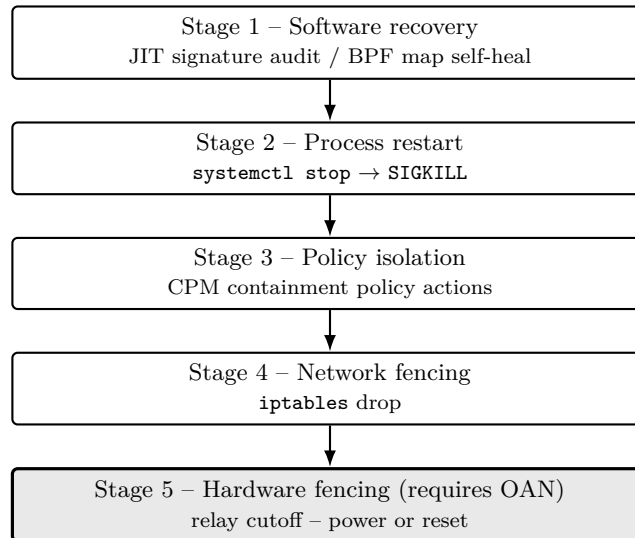


Figure 7: The recovery ladder. Stages 1–4 are entirely within **kb-checker**'s existing software authority; only Stage 5 needs OAN, and only after every stage above it has already failed.

Because Stage 5 sits strictly below the existing chain, a healthy host never reaches OAN’s relay at all — the four software stages above it already resolve the overwhelming majority of incidents on their own.

13 Scaling to a Fleet: the Shared Chassis Design

A single OAN unit protects a single host. Most real deployments protect more than one machine, so OAN’s reference architecture is a **shared chassis**: one physical box that supervises up to **five hosts** at once, capped deliberately rather than grown further.

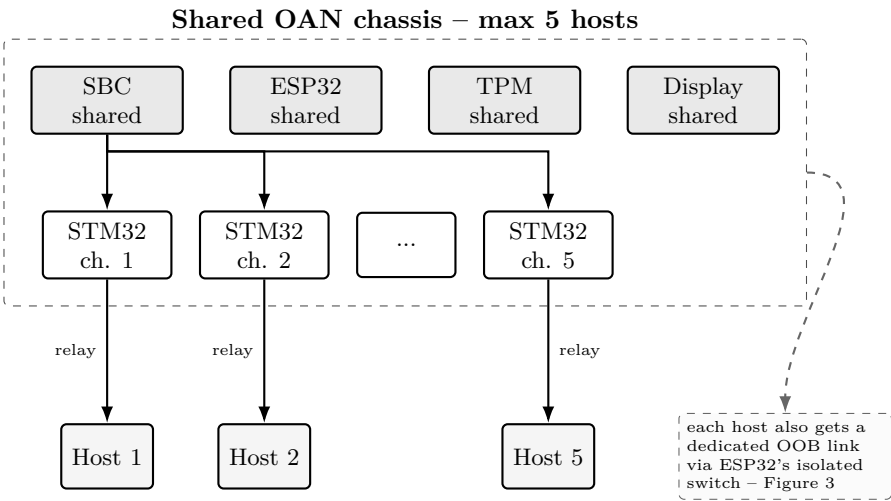


Figure 8: One shared chassis, up to five hosts. Each STM32 channel’s relay wire is the fencing path; the OOB attestation link to each host runs separately through ESP32’s isolated switch, detailed in Figure 3.

13.1 What Is Shared, and What Every Host Still Needs

Sharing the chassis removes *redundant* hardware, not per-host hardware — fencing and attestation are inherently host-specific, so every host still needs its own physical wiring.

Shared once, across up to 5 hosts	Always required, per host
SBC (attestation engine, policy, dashboard)	A relay tap wired into that host’s own power/reset header
ESP32 (owns the isolated switch port)	A dedicated OOB link endpoint — a small dongle/NIC distinct from the host’s normal network card
TPM (root of trust)	The STM32 channel logic watching that one link/relay pair
Display	Wiring and connectors specific to that host

The transport for each host’s channel is a dedicated, physically isolated Ethernet segment — its own small unmanaged switch, no uplink to the production LAN — rather than loose serial

cables. This is exactly the pattern enterprise BMC and IPMI [22] management networks already use at this scale, for the same reason: physically and logically separate from anything the host’s own kernel drives. A VLAN carved out of the host’s *own* NIC does not count as out-of-band, because the host’s kernel still drives that interface and can forge or block traffic on it; each host’s OOB link terminates on hardware distinct from its main NIC.

13.2 Why Cap at Five Hosts

- **Blast radius.** If the shared chassis itself fails or is physically tampered with, only five hosts lose OOB coverage at once, not an entire lab.
- **Commodity part sizes.** 5–8 channel relay/PDU boards and 5–8 port unmanaged switches are natural off-the-shelf sizes. Going bigger means custom boards instead of commodity ones.
- **USB/GPIO fan-out on the SBC.** A single-board computer’s USB and GPIO budget gets cramped well before ten-plus channels; five keeps the shared SBC within what a stock board can drive without a custom PCB.

A lab of thirty machines is served by **six** shared chassis, each independently reporting up to the fleet layer (§14) — not one giant chassis, and not thirty individual units.

14 Fabric Management: One Node to a Fleet

The Fabric Management Service (FMS) is the software layer, running on the SBC (§7.1), that turns many independent chassis into one coordinated **Fabric**. It organizes protected systems into a clean hierarchy.

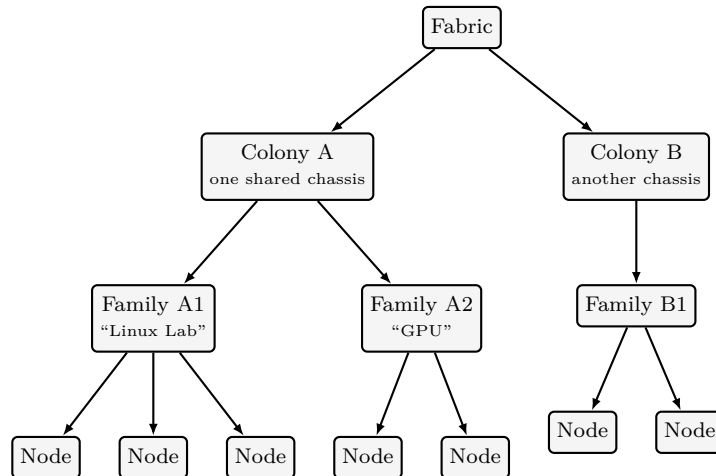


Figure 9: Fabric → Colony → Family → Node. A Colony maps directly onto one shared chassis (§13); Families let administrators group hosts by purpose regardless of which chassis they are physically wired to.

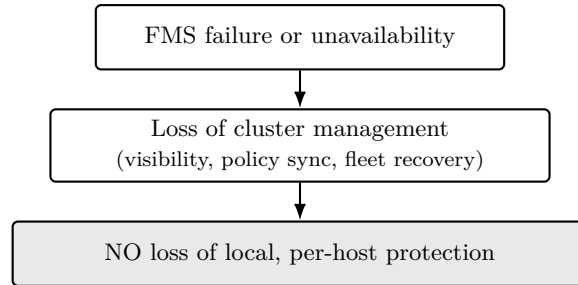
Term	Meaning
Fabric	The entire managed deployment — every Colony, everywhere.
Colony	One shared chassis and the up-to-five hosts wired to it: a lab bench, a rack, a department.
Family	A logical grouping of Nodes by purpose (“AI Family”, “Database Family”), independent of physical wiring.
Node	One protected host, running the full KB stack.

14.1 The Local Autonomy Principle

Local Autonomy Principle

Every KB node remains fully capable of protecting itself without FMS.

FMS is never part of the runtime security path. Without it, every node still has **kb-core**, **kb-control-plane**, **kb-checker**, local OAN attestation, local recovery, and local fencing — nothing about single-host protection changes. With FMS, the fleet additionally gains visibility, policy synchronization, fleet-wide recovery coordination, and a single inventory. Losing FMS costs coordination, never protection:



This is why FMS is classified *operationally important*, *not security-critical*, and why it runs entirely on the Fabric Management Plane (§6), never on the OOB Trust Plane. An FMS outage is an inconvenience; it is never a hole in a host’s defenses.

14.2 What Crosses the Fabric Plane

FMS never receives raw kernel state. Each node reports a **TPM-signed summary**, produced by the same trust chain described in §8 — health, alerts, inventory, version, and integrity status, and nothing more:



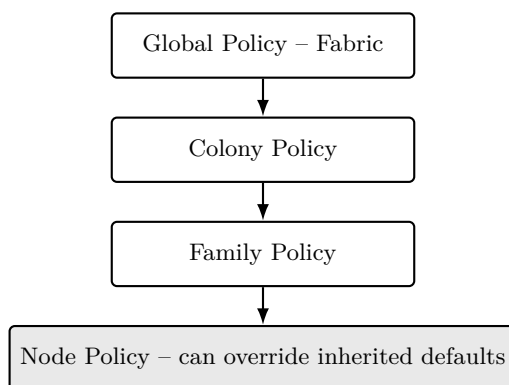
Because that summary is signed against the same TPM that already seals both OAN’s own state and the host’s attestation (§8), FMS can trust what it receives without ever needing raw access to anything privileged — and a compromised Fabric Management Plane still cannot forge a clean report for a host that is actually compromised.

14.3 Fleet Responsibilities

Capability	What it does
Fabric Discovery	Finds new KB deployments automatically: registration, hardware fingerprinting, platform and version detection.
Node Registry	Keeps the inventory — node ID, hostname, platform, kernel version, family, colony, health, trust status, last heartbeat.
Membership Management	Join/leave Fabric, node removal, family migration, colony reassignment.
Health Monitoring	Host availability, resource usage, <code>kb-core</code> / <code>kb-checker</code> /OAN connectivity, heartbeat.
Policy Management	Define a security policy once; deploy it to a Fabric, a Colony, a Family, or a single Node.
Event Aggregation	Unified timeline across every node — search, indexing, historical record.
Cross-Host Correlation	Surfaces shared indicators and simultaneous anomalies across the fleet, in cooperation with <code>kb-aads</code> .
Recovery Coordination	Multi-node quarantine, fabric-wide recovery, controlled restart ordering.
Deployment Management	Version inventory, compatibility checks, controlled rollout.

14.4 Policy Inheritance

Policy flows downward through the hierarchy, and a more specific level can always override a broader default:



Multiple Colonies never share a single point of failure at the trust layer: each Colony keeps its own OAN chassis and its own FMS instance, with no global master coordinating between them. A Colony stays autonomous even if OANs are later given a way to synchronize with each other for visibility — autonomy is a property of the trust layer, not a limitation on convenience features built above it.

15 Deployment Walkthrough: From One Desk to a Lab

The shared-chassis design (§13) is built so that a single chassis grows smoothly from protecting one machine to protecting five, without ever touching the shared hardware again once it is set up. This section walks through that growth path in the order a team would actually experience it.

Stage	What happens	What already exists, reused as-is
1	Assemble one shared chassis: SBC, ESP32, TPM, display, and the dedicated OOB switch.	Nothing yet — this is the one-time setup.
2	Wire host #1: relay tap into its power/reset header, dedicated OOB dongle, one STM32 channel. Run the attestation heartbeat (§10) end to end.	The entire shared chassis from Stage 1.
3	Add hosts #2 and #3: repeat only the per-host wiring — another relay tap, another dongle, another STM32 channel.	SBC, ESP32, TPM, display, switch — unchanged.
4	Fill the chassis: add hosts #4 and #5 the same way. The chassis is now at its Pentagon cap (§13).	Same shared hardware, still unchanged since Stage 1.
5	Organize with FMS: group the five Nodes into Families by purpose, set Colony-level policy, and start collecting the signed-summary event stream (§14).	All hardware from Stages 1–4; this step is pure configuration.
6	Scale past five: stand up a second, independent chassis for hosts #6–#10, reporting to the same Fabric.	A second copy of Stage 1’s shared hardware — deliberately <i>not</i> an extension of the first chassis (§13’s cap rationale).

In Plain Terms

Notice what never appears in the “what happens” column after Stage 1: buying another SBC, another TPM, or another display. Every stage from 2 through 5 is wiring and configuration on top of hardware that was bought exactly once. That is the whole economic argument for the shared-chassis design, told as a sequence of actions instead of a cost table (§16 has the numbers).

16 Building One: Bill of Materials

The numbers below describe a single shared chassis (§13) built from commodity, off-the-shelf parts — no custom silicon, no board fabrication required to get a working unit on a bench. All INR figures use an approximate ~INR 87/USD conversion for scoping; real domestic sourcing typically comes in lower.

16.1 Shared Chassis Parts (bought once per chassis)

Component	Purpose	USD	INR
SBC (Raspberry Pi 4/5 or Intel N100)	Attestation engine, policy, dashboard, FMS	\$35–80	INR 3,050–6,950
microSD card (32GB+)	OS + watchdog binary	\$8	INR 700
ESP32 dev board	Management/comms processor	\$6–10	INR 525–875
Discrete TPM 2.0 module	Hardware root of trust	\$20–40	INR 1,750–3,500
I2C OLED/LCD display	Live status readout	\$5–10	INR 440–875
Shared total, per chassis		\$74–148	INR 6,440–12,880

16.2 Per-Host Channel Parts

Component	Purpose	USD	INR
STM32 dev board	That host’s safety controller/relay channel	\$10–25	INR 870–2,175
Relay channel	Fencing for that host, on a shared multi-channel board	\$6	INR 520
OOB link hardware	Dedicated dongle/NIC, distinct from the host’s main NIC	\$8	INR 700
Wiring/connectors	Cable run to that host	\$8	INR 700
Total, per host		\$32–47	INR 2,785–4,090

16.3 Total Cost by Fleet Size

Config	Hosts	Chassis	Total (USD)	Total (INR)
Single	1	1	\$106–195	INR 9,220–16,965
Triple	3	1	\$170–289	INR 14,790–25,145
Pentagon (cap)	5	1	\$234–383	INR 20,360–33,320
Lab	30	6	\$1,404–2,298	INR 1,22,150–1,99,925

In Plain Terms

The shared parts (SBC, ESP32, TPM, display) are bought exactly once per chassis, whether it is serving one host or all five. Every host past the first only adds the \$32–47 per-host line — which is why Pentagon’s *per-host* cost (\$46.80–76.60) ends up under half of Single’s (\$106–195), even though Pentagon’s *per-person* cost is higher. A team actually protecting more than one machine gets more coverage per dollar by filling a chassis than by building single, standalone units.

17 Building It on a Budget

The parts list above uses reasonable retail defaults, not the cheapest possible build. A budget-constrained team can bring the shared-chassis cost down substantially by swapping specific parts — without touching OAN’s architecture or its guarantees.

Component	Reference	Budget	Swap
SBC	\$35–80	\$15–20	Pi Zero 2 W handles the workload comfortably
ESP32	\$6–10	\$3–5	Bare WROOM module, no breakout board
STM32	\$10–25	\$3–6	Bare “Blue Pill” clone
Relay	\$6/ch	\$4–6/ch	Shared multi-channel board
OOB dongle	\$8	\$3–5	Generic CP2102 clone
Display	\$5–10	\$0	Dropped entirely – optional, non-decisional

This brings a Pentagon (5-host) chassis down to roughly **\$114–168 / INR 9,918–14,616** total — about \$11.40–16.80 per person for a team of ten.

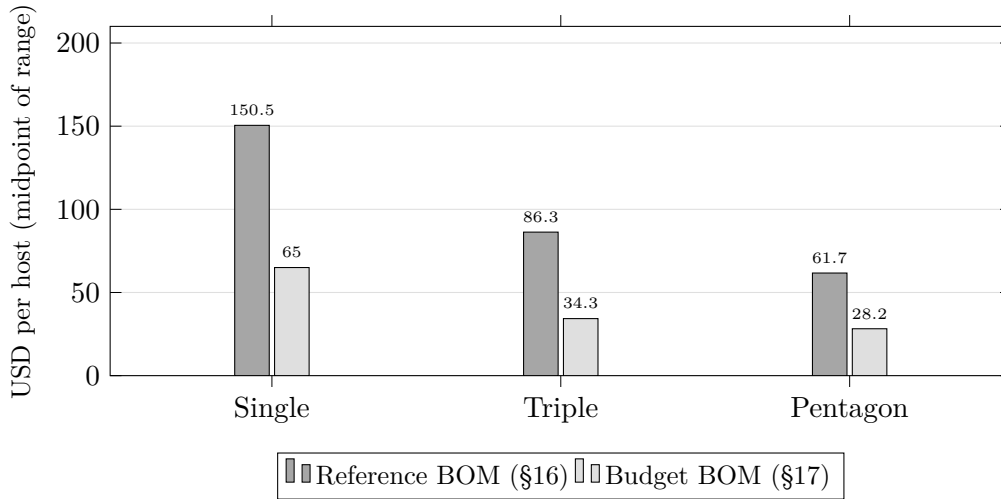


Figure 10: Per-host cost falls as a chassis fills, at both budget tiers, because the shared SBC/ESP32/TPM/display cost is the same regardless of how many of the five host slots are in use (§16’s “Marginal Cost of Filling a Chassis” logic).

What Stays, No Matter the Budget

Two line items are never on the table, at any budget:

- **The discrete TPM.** It is the one component whose entire job is being the thing a compromised host cannot fake. A software firmware-TPM would put OAN’s own root of trust on the same silicon as its general-purpose SBC — reintroducing, inside OAN itself, the exact same-failure-domain problem OAN exists to solve for the host.
- **The STM32 split.** Folding the watchdog and relay logic back onto the SBC to save a few dollars would recreate the “the watchdog shares Linux’s failure domain” problem §7.2 exists specifically to avoid.

18 The Production Appliance

Beyond the bench-buildable chassis, OAN’s production form factor is a single enclosed appliance:

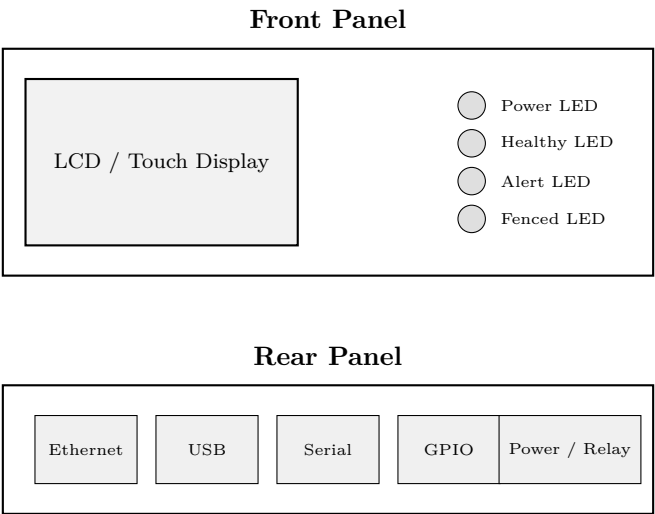


Figure 11: Front panel gives a technician an at-a-glance read of host state; rear panel exposes every physical interface described in §5–§8. Neither panel is on the trust-critical path – both are read-outs and connectors, never decision inputs.

Front panel	Rear panel
LCD / touch display	Ethernet
Power LED	USB
Healthy LED	Serial
Alert LED	GPIO
Fenced LED	Power, relay connector

Internally, the SBC, STM32, ESP32, TPM, relay board, cooling, and power regulation are all enclosed as one hardware unit – the same architecture as the bench chassis (§13), packaged for a rack or a desk rather than loose dev boards on a breadboard.

19 Why This Architecture Works

Every component in OAN has exactly one job, and no component both makes the security decision and runs the general-purpose networked operating system:

Component	Responsibility
SBC	Decision-making, attestation, fleet management
STM32	Deterministic safety, watchdog, relay control
ESP32	Management communication and provisioning
TPM	Hardware trust anchor for both OAN and the host
Relay	Physical host fencing
Display	Local, read-only operational interface

That separation is what lets OAN make a claim no purely-software watchdog can: even if the SBC crashes, even if the ESP32 is fully compromised over the network, and even if the host’s own kernel is lying about everything it reports, the STM32 still keeps its clock, the TPM still keeps its keys, and the relay still works.

20 Security Properties and Honest Limits

A design is only as trustworthy as its stated boundaries. This section makes both halves explicit: what OAN guarantees, and what it deliberately leaves to other parts of Kernel Borderlands.

20.1 What OAN Guarantees

Guarantee	Why it holds
Detection of a fully compromised host kernel	The heartbeat (§10) never routes through the host’s kernel to reach OAN – there is no code path for a compromised kernel to intercept.
Fencing that does not require host cooperation	The relay is wired to the STM32, which never runs code the host can influence (§7.2).
Tamper-evidence of OAN’s own state	Sealed against the TPM, covering both OAN’s own boot state and the host’s reported measurements (§8).
Fleet coordination that cannot weaken any single host	The Local Autonomy Principle (§14.1) makes this true by construction, not by discipline.

20.2 What OAN Does Not Do

- **It does not detect intrusions.** That is `kb-core`, `kb-control-plane`, and `kb-aads`’s job. OAN only verifies that the components doing that job are themselves still honest – it has no opinion on process behavior, network traffic, or file activity.
- **It does not protect against physical access to itself.** A chassis that an attacker can physically open, rewire, or replace is a different threat model (physical security of the room), outside OAN’s own scope.
- **It does not eliminate the need for the four software recovery stages.** OAN is Stage 5 of five (§12) – the overwhelming majority of real incidents are expected to resolve at Stages 1–4, long before OAN’s relay is ever involved.

- **It does not make the Fabric Management Plane trust-critical.** FMS coordinates; it never authorizes (§14).

In Plain Terms

A useful one-line summary: OAN answers exactly one question – “can I still trust what this host is telling me?” – and answers it in a way a fully compromised host cannot fake. It intentionally does not try to answer any other question.

21 What-if: Boot/Rootkits?

Every guarantee in §20 rests on one witness: **kb-checker**. So the sharpest question this whole design invites is the one this section answers: *what if kb-checker itself is lying?* Not killed, not silent – compromised, and still talking.

21.1 The Question, Stated Precisely

Analogy

A courtroom only works if the witness is telling the truth. A locked door on the witness stand doesn’t help if someone has already gotten to the witness before they walked in. OAN’s heartbeat (§10) is that witness stand – and up to this point, every guarantee this document has made assumes the witness is honest. This section is about what happens the moment that assumption breaks.

If an attacker achieves full kernel (Ring 0) compromise, they are not outside **kb-checker**’s world – they are underneath it. Every fact **kb-checker** observes – process lists, loaded modules, BPF map contents, `/proc` entries – ultimately comes from the same kernel the attacker now controls. That opens two structurally different failure modes, and the design so far has only answered one of them well.

Silence is already solved, structurally, by §10: the STM32’s liveness timer never took input from the host to begin with, so it doesn’t need to know *why* the heartbeat stopped, only that it did.

Lying is harder, because a compromised kernel can feed **kb-checker** forged answers to the exact syscalls it uses to check on **kb-core** and itself, and **kb-checker** – running its real, unmodified code – will faithfully sign and send an attestation that is, from its own vantage point, completely honest. This is a confused-deputy problem [2]: **kb-checker**’s *code integrity* can be real while its *inputs* are poisoned below the layer it can see.

The Limit That Doesn’t Move

None of what follows makes **kb-checker** trustworthy again once the kernel beneath it is compromised – that is not achievable in software, and this document will not pretend otherwise. What follows narrows the set of attacks that can succeed *without leaving a detectable trace*, using components already in this design, at zero additional hardware cost. The goal shifts from “prevent the lie” to “make the lie expensive, and turn a failure into evidence.”

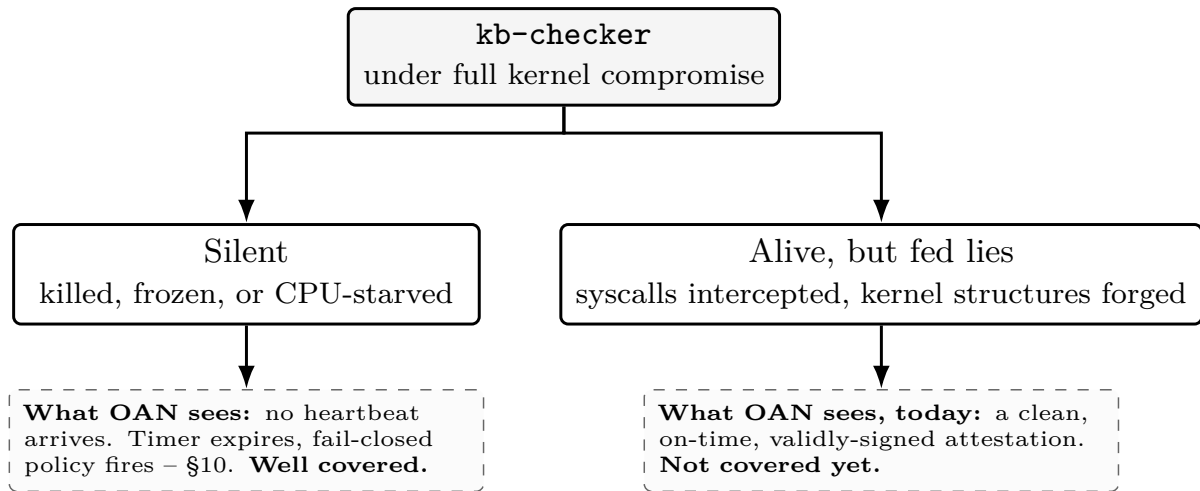


Figure 12: Two structurally different failures hiding behind one name. Silence is a solved problem – the STM32’s timer doesn’t care why the heartbeat stopped, only that it did. A **kb-checker** that stays alive and keeps signing, fed false inputs by the kernel underneath it, is the harder case this section is about.

21.2 Already Closed: Self-Containment

One piece of this is already done, in the codebase, not just on paper. The Critical Process Module’s protected-component floor previously covered **kb-sensor**, **kb-agent**, **kbctl**, **policy-engine**, and **dashboard** – but not **kb-checker**. That meant a malfunctioning or manipulated detection engine could get **kb-checker** itself recommended for containment through KB’s *own* pipeline, with nothing to stop it. **kb-checker** (and **kbd**, which also covers CPM’s “policy-engine” since policy evaluation runs inside the **kbd** process) are now in CPM’s compiled-in floor, with real, documented install paths.

This closes a real gap, but it is worth being precise about which one: CPM is an authorization gate in front of KB’s *own* containment pipeline (§2.4 of **CPM.md**, “no self-containment, ever”). A kernel-resident rootkit that never asks **contained_pids_map** for permission – because it corrupts scheduler state or task structures directly – never touches the path CPM guards. What it *does* buy: the “silent” bucket in Figure 12 no longer has “KB’s own logic turned on itself” as a candidate cause. Silence now points more specifically at something external.

21.2.1 Hardening CPM’s Own Data Structures

CPM’s authorization gate is only as trustworthy as the BPF maps it reads its decisions from. Both are now locked down at the map level, not just guarded by convention:

- **BPF_F_RDONLY_PROG** on **contained_pids_map** and **protected_exec_paths_map**. This flag restricts writes to the syscall boundary – no in-kernel BPF program can write either map, only userspace can, over the existing **handle_incoming_containment_cmd()** and **read_cpm_protected_exec_from_bridge()** paths. A compromised or buggy BPF program – a materially harder bar than a userspace exploit, given the verifier, but not an impossible one – can read CPM’s protected list but cannot forge an entry into it or clear **contained_pids_map** from the kernel side.
- **bpf_map_freeze()** on **kb_sensitive_paths**. Stronger than the flag above: freezing makes

a map permanently read-only, from *both* kernel and userspace, for the rest of that boot. Applied only to the one map that is genuinely done being written once startup finishes – placed after the compiled-in floor populates it *and* after the one-shot merge from `kbd` completes, immediately before the BPF programs attach.

Why Not Freeze Everything

`protected_exec_paths_map` and `protected_workload_paths_map` stay unfrozen deliberately. Both take legitimate writes *after* startup – an operator-pushed overlay for the former, CWP’s live workload-registration wire push for the latter – and freezing either would silently break a real feature rather than close a gap. The distinction only holds if it’s checked against each map’s actual write call sites individually; assuming “looks similar, freeze it too” is exactly the mistake this was built to avoid. `bpf_map_freeze()` is also a best-effort step, not a fatal one: if the freeze call itself fails, the sensor logs a warning and continues unfrozen rather than refusing to start – a hardening step should not become a new availability risk on its own.

21.3 Getting the Key Out of Host Memory

Before anything about detecting a lie: there is a sharper, more immediate version of this problem worth closing first. If `kb-checker`’s OOB signing key sits in its own process’s RAM, a Ring-0 attacker doesn’t need `kb-checker` alive and lying at all. They can read the key straight out of memory and forge attestations directly – or simply kill `kb-checker` and impersonate it from then on, forever, with no watchdog left to notice.

Which TPM

Every mechanism in this and the next subsection uses the *host’s own* TPM 2.0 – physically on the protected machine, not OAN’s. A TPM can only make a hardware-backed statement about the machine it is part of; OAN’s own TPM (§8) can seal OAN’s own integrity and verify what `kb-checker` reports, but it cannot attest to the host’s measured-boot chain from across the OOB link – there is no hardware path between them. Essentially every modern server or workstation already ships with one, discrete or firmware (Intel PTT [20], AMD fTPM [21]), for Secure Boot or disk encryption. Zero new hardware, and no tension with `oan-cheap.md` §3.7’s rejection of fTPM: that argument is specifically about not substituting the host’s fTPM for *OAN’s own* independent trust anchor. Using the host’s already-present TPM for a host-local job is a different question, and this is precisely the case where a firmware TPM is exactly what `oan-cheap.md` itself calls “genuinely free.”

This is a protocol decision, not a bill-of-materials change – the host’s own TPM is already there on essentially any real deployment target. `kb-checker` asks it to sign; the private key never exists in host RAM at all. But look closely at the right-hand side of Figure 13: a root-level attacker can still simply place the *same request* `kb-checker` would have made. The TPM, by itself, doesn’t know or care who is asking – it only knows what key was asked for. Key-in-TPM stops *theft*. It does not, on its own, stop *impersonation*. That requires one more piece.

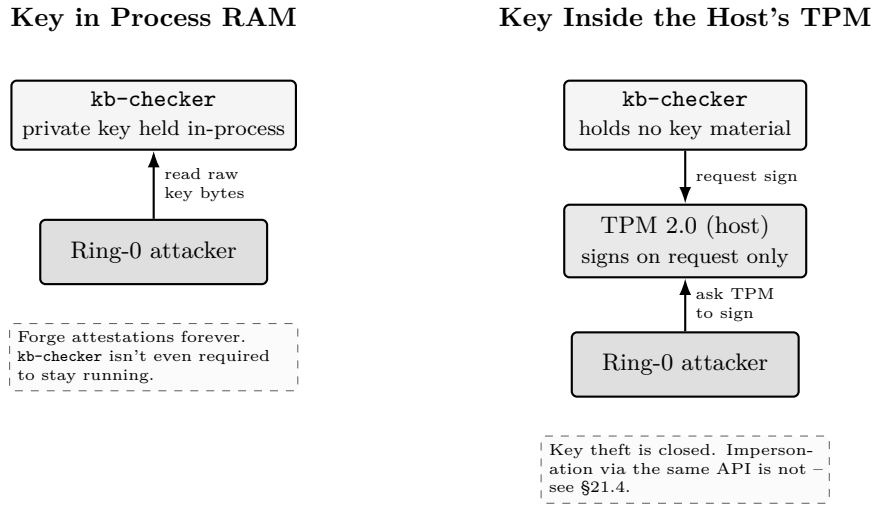


Figure 13: The host’s own TPM 2.0 – already present on essentially any modern server or workstation – closes passive key theft for free: have **kb-checker** request signing *operations* rather than holding raw key material. Zero new hardware; a protocol decision.

21.4 The Critical Piece: Sealing Key Usage to Measured Boot State

This is the one mitigation in this section that doesn’t just raise the cost of a same-privilege-level attack – it removes the attacker’s ability to use the key at all, for an entire class of compromise, without **kb-checker** needing to notice anything.

21.4.1 The Mechanism

TPM 2.0 Platform Configuration Registers (PCRs) [4, 5] are not writable – only *extendable*: each measurement folds into the register as $\text{PCR}_{\text{new}} = H(\text{PCR}_{\text{old}} \parallel \text{measurement})$. This makes every PCR a one-way, cumulative hash chain rooted at boot. Firmware measures itself, then measures the bootloader; the bootloader measures the kernel image and `initrd` before executing them; and, at runtime, the Integrity Measurement Architecture (IMA) [6, 7] – standard Linux, no new hardware – extends a PCR every time a kernel module loads.

A key can be created with a `TPM2_PolicyPCR` authorization policy: a digest computed over chosen PCR values at seal time. From then on, the TPM performs a signing operation with that key only if the *current* PCR values, hashed the same way, match that digest – checked entirely inside the chip. There is no step in that sequence a same-privilege-level attacker can intercept, because the attacker is not inside the TPM.

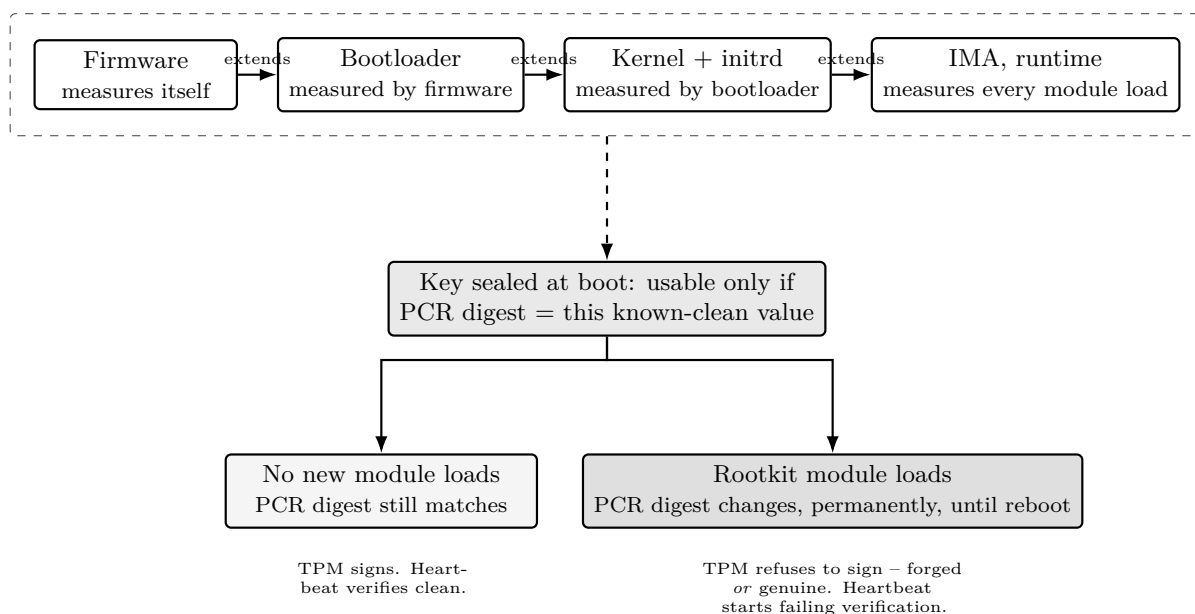


Figure 14: The measured-boot chain, and what a rootkit module load does to it. PCRs only extend, never reset, short of a full platform reboot through the same chain – so a rootkit cannot simply “fix” the value from inside the kernel it just compromised.

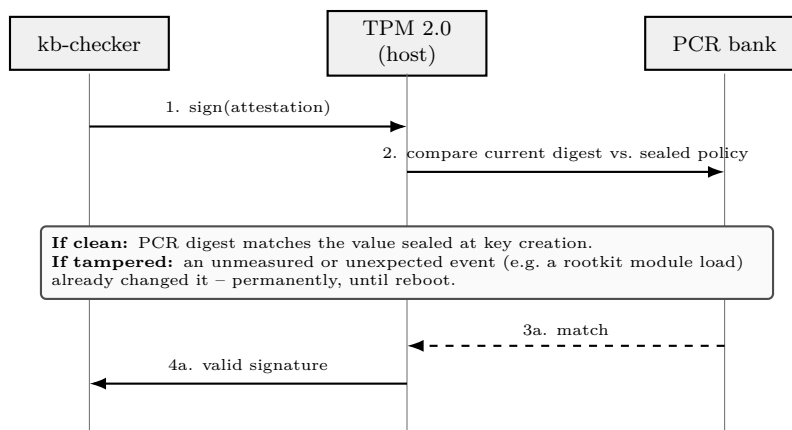


Figure 15: The PCR-sealed signing request. Step 2 happens inside the TPM chip itself – the host never gets to answer “is my boot state clean,” it only gets told the result. On mismatch, the TPM returns an error instead of step 4a, for *any* caller, forged or genuine.

21.4.2 Before and After, From the Attacker’s Side

Key custody	What the attacker needs
Key in process RAM	Any Ring-0 primitive, from any privilege-escalation class. Read the key, forge forever.
Key in TPM, unsealed	Any Ring-0 primitive, plus the ability to call the TPM’s signing API – trivial once root. Theft is closed; impersonation is not.
Key in TPM, PCR-sealed	Ring-0 achieved via a path that never extends the sealed PCRs – narrowed from “any privilege escalation” down to memory-corruption-only exploitation of <i>already-loaded</i> , <i>already-measured</i> code, with zero new modules, for as long as the attacker wants to keep forging.

In Plain Terms

The important property isn’t just “harder” – it’s **self-terminating**. A rootkit that needs to load anything, even something auxiliary, even temporarily, doesn’t get an error and a chance to route around it while staying resident. The key goes dark for the rest of that boot session, full stop, the moment the PCR extends. Most real persistent Linux rootkits ship as loadable modules, because writing durable kernel-level capability without ever loading new code is a substantially harder engineering problem than the alternative – so this closes the well-trodden path, not a rare one.

This is also what turns a bare fencing event into a specific claim (§21.9): if the TPM starts refusing to sign, that is not “something is wrong, generically.” It is “the measured-boot state changed since this key was sealed” – about as close to a direct rootkit or bootkit signature as this design can produce without new hardware.

21.4.3 Demo Environment: Trying This on Real Hardware

This mechanism is testable on unmodified, real hardware – most laptops and desktops built in the last several years already have everything it needs.

Checking for a TPM, on Linux:

```
ls /dev/tpm* # /dev/tpm0, /dev/tpmrm0 present?
cat /sys/class/tpm/tpm0/tpm_version_major # should print 2
dmesg | grep -i tpm # kernel’s own boot-time detection
```

If nothing shows up, the chip may still exist but be disabled in firmware (look for “Intel PTT” or “AMD fTPM”/“Security Device” in BIOS/UEFI setup [25]) – a firmware setting, not a Linux one, so it needs a reboot into setup, not a kernel config change. Install the tooling needed for everything below:

```
sudo pacman -S tpm2-tools # Arch and derivatives
sudo apt install tpm2-tools # Debian/Ubuntu
```

`tpm2-tools`, from the `tpm2-software` project [28], is the standard userspace CLI for TPM 2.0.

Before Touching Anything

Check first whether the machine already relies on the TPM for disk encryption (`systemd-cryptenroll`, Clevis+Tang, or similar) – creating or evicting TPM objects could interfere with an existing unlock path. If none of that is in use, there is nothing to worry about. Either way, the walkthrough below deliberately never touches PCRs 0–10, the ones any real boot chain (firmware, bootloader, kernel, IMA) actually measures into – it uses PCR 16, one of a small number the TCG spec [32] defines as resettable “debug” registers, untouched by any standard Linux boot and resettable without a reboot. Confirm it is at its expected all-zero baseline first:

```
tpm2_pcrread sha256:16
```

The walkthrough. Seal a secret to PCR 16’s current value, confirm it unseals cleanly, then extend that PCR the way a rootkit module load would extend a real one, and watch the same unseal fail – §21.4’s whole argument, live, without touching a real kernel module or the boot chain the machine actually depends on:

```
# 1. Primary key, owner hierarchy
tpm2_createprimary -C o -c primary.ctx

# 2. Trial session to compute the policy digest for "PCR 16 unchanged"
tpm2_startauthsession -S session.ctx
tpm2_policypcr -S session.ctx -l sha256:16 -L policy.digest
tpm2_flushcontext session.ctx

# 3. Seal a test secret under that policy
echo -n "test-secret" | tpm2_create -C primary.ctx -u seal.pub \
    -r seal.priv -L policy.digest -i-
tpm2_load -C primary.ctx -u seal.pub -r seal.priv -c seal.ctx

# 4. Real policy session, satisfy it, unseal -- should print "test-secret"
tpm2_startauthsession --policy-session -S session.ctx
tpm2_policypcr -S session.ctx -l sha256:16
tpm2_unseal -c seal.ctx -p session:session.ctx
tpm2_flushcontext session.ctx

# 5. Extend PCR 16 -- stand-in for an unmeasured/unexpected boot event
tpm2_pcrextend 16:sha256=$(echo -n "rootkit" | sha256sum | cut -d' ' -f1)

# 6. Same unseal, fresh session -- now fails
tpm2_startauthsession --policy-session -S session2.ctx
tpm2_policypcr -S session2.ctx -l sha256:16
tpm2_unseal -c seal.ctx -p session:session2.ctx
tpm2_flushcontext session2.ctx

# 7. Clean up -- PCR 16 resets without a reboot, unlike 0-15
tpm2_pcrreset 16
```

Step 6 is the demonstration: identical request, identical secret, identical key – the only thing that changed is the PCR value, and the TPM refuses regardless of whether the request coming in is the legitimate caller or an attacker replaying the exact same commands. Exact flag names can shift slightly between `tpm2-tools` releases; if a step errors, `man tpm2_<command>` for the installed version usually resolves it in one look.

What This Does Not Close

- **Memory-corruption-only exploits.** An attack path that reaches Ring 0 purely through in-place patching of already-loaded, already-measured code – no new module, ever – extends no PCR and trips nothing. Narrower than “any rootkit,” not zero – and narrower still once §21.5’s mitigations are stacked on top.
- **Physical TPM bus sniffing.** A discrete TPM communicating over an unencrypted LPC/SPI bus is a documented attack class (the same category demonstrated against BitLocker on some hardware). It requires physical board access, not a remote exploit – TPM 2.0 parameter-encryption sessions mitigate it, at zero new hardware cost, same as everything else in this section.
- **Upstream compromise of the measurement chain itself** (firmware, bootloader, Secure Boot bypass) is a different, higher-effort threat class this doesn’t address – and isn’t really OAN’s problem to solve; it’s the layer OAN’s own trust starts from.
- **Real operational cost.** Every legitimate kernel update or intentionally-added module requires re-sealing the key against a new expected PCR digest. That is a key-management process that has to actually exist and be exercised correctly – skip it once and a routine patch locks the key out for no security reason; build a habit of re-sealing without checking *why* the PCR changed, and the whole mechanism is quietly defeated.

21.5 Narrowing the Memory-Corruption Gap

PCR-sealing leaves exactly one gap open: an attacker whose path to Ring 0 never triggers a measured event. That isn’t one attack, it’s a category – and most of that category can be narrowed further with mitigations that already ship in mainline Linux and mainstream compilers, at zero new hardware cost. None of them close the gap completely; nothing can. Each removes one specific technique from the attacker’s toolbox, and stacked together, what survives is a much narrower thing than “any memory-corruption bug.”

21.5.1 Preventive Hardening, Already Mostly Free

ASLR/KASLR [10] randomizes where the kernel and its structures land in memory on every boot. A memory-corruption bug that doesn’t know where anything is usually just crashes the machine rather than achieving reliable code execution. **KPTI**’s role here is more specific than it first looks: its actual job is closing Meltdown-class speculative-execution reads of kernel memory from userspace [17] – and one of the most practical uses of that class of leak is defeating KASLR before ever triggering the corruption bug [18]. KPTI’s contribution to this problem is indirect but real: it closes a common way an attacker turns “crash” into “reliable exploit.”

Stack canaries [11] (`-fstack-protector` and family) place a random value next to the return address on the stack; a stack buffer overflow overwrites the canary before it reaches anything that matters, and the check fails before the corrupted return address is ever used. Scope-limited, precisely: this defends stack overflows only, not heap corruption, use-after-free, or type confusion. A stronger, hardware-backed version exists – Intel CET’s shadow stack (**SHSTK**) [12] keeps a second, hardware-protected copy of return addresses a regular overflow cannot touch – but that specific guarantee needs a CET-capable CPU. A software-only shadow stack exists too, and is weaker; the two should not be conflated.

CONFIG_STRICT_KERNEL_RWX enforces W^X (write XOR execute) [13] on kernel memory: once loaded, code pages are read-only and executable, data pages are non-executable. This directly

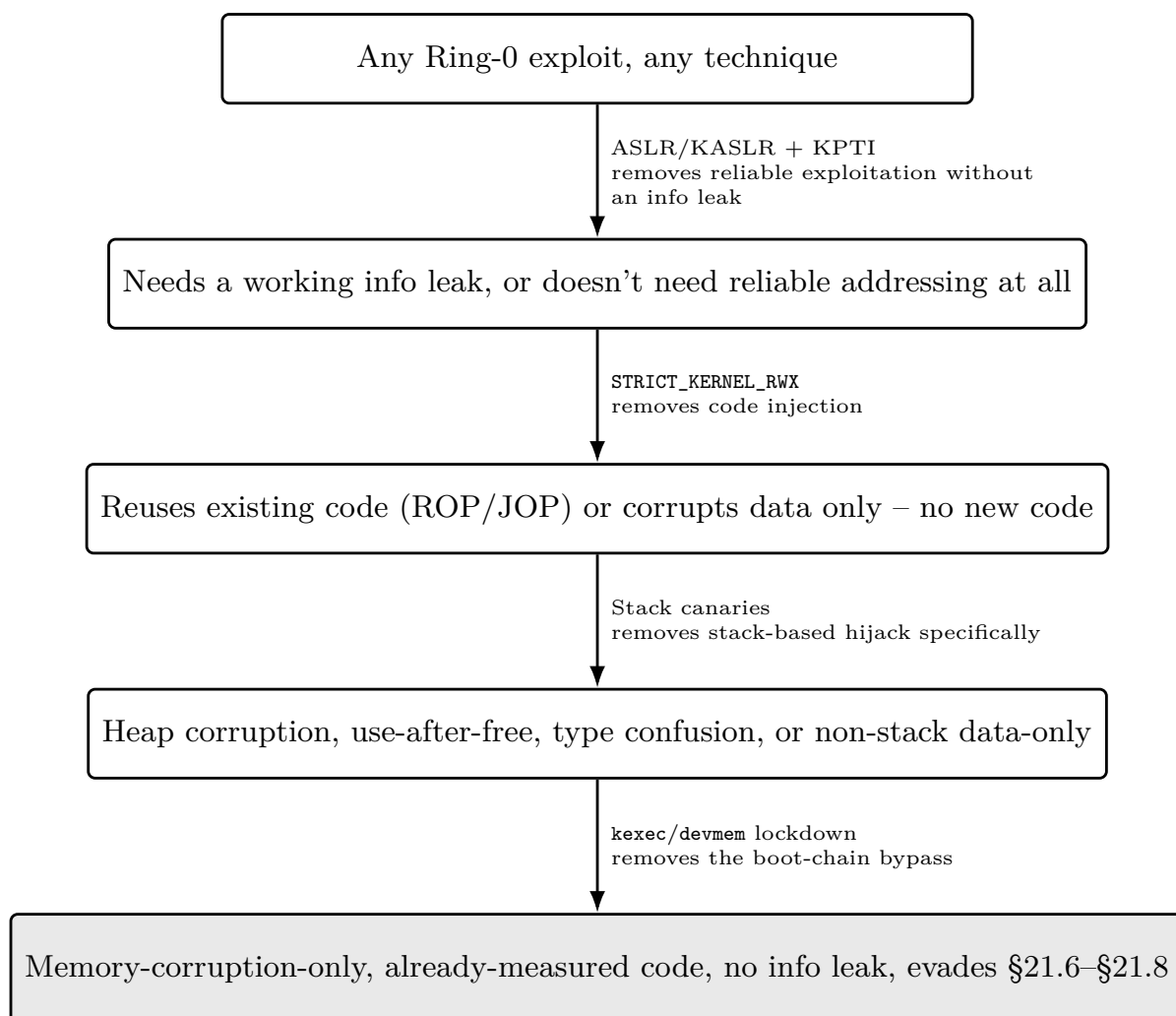


Figure 16: What’s left after every free mitigation is stacked. Each layer removes one specific technique, not “exploitation in general” – the box that survives at the bottom is deliberately still non-empty.

blocks the classic “corrupt memory, inject shellcode, jump to it” technique [14]. It is precisely scoped, and precisely limited: **return-oriented** [15] and **jump-oriented programming (ROP/JOP)** [16], and **data-only attacks, inject no new code at all**. They redirect control flow to reuse code that is already legitimately executable, or corrupt data structures – credentials, function pointers – without touching a code page. W^X is silent on both. This is the sharpest technique that survives every mitigation on this list at once, and it is what Figure 16’s bottom box is built from.

Compatibility, Not Just Security

CONFIG_STRICT_KERNEL_RWX can break legacy or out-of-tree kernel modules that expect to self-modify code at runtime. This is a one-time compatibility check against your actual driver set at deployment, not an ongoing operational risk – unlike the trade-off in §21.5.4 below.

21.5.2 Confirmed on kb-core’s Own Userspace Daemon

The mitigations above are described in general terms; kb-core’s own userspace loader/bridge – `kbd_sensor` and its six collector binaries, the code that manages CPM’s BPF maps (§21.2.1) and loads the eBPF programs its LSM hooks run as – now actually carries them, confirmed against the compiled output rather than assumed from the build flags:

- `-fPIE/-pie`: the userspace equivalent of KASLR above, applied to all 7 of kb-core’s build targets.
- `-fstack-protector-strong` and `-fstack-clash-protection`: stack canaries, plus protection against stack-clash allocation attacks – the latter not covered anywhere above.
- `-D_FORTIFY_SOURCE=3`: compile- and runtime bounds checking on common libc calls (`memcpy`, `sprintf`, and similar), including cases where the object size is only known at runtime – the strongest variant, requiring `-O1` or higher to take effect.
- `-Wl,-z,relro,-z,now`: full RELRO. Resolves every dynamic symbol at load time and marks the GOT read-only immediately rather than lazily – the userspace analog of `CONFIG_STRICT_KERNEL_RWX`’s protection, applied to a different writable target (the GOT, not kernel code pages) against the same class of overwrite-and-redirect technique.

Confirmed against the actual binary, not just the Makefile: `readelf -h` for `ET_DYN` (PIE), `readelf -d` for `BIND_NOW`, and `nm` for `__stack_chk_fail`, after a full `make clean && make`. A flag that looks correct in a Makefile and doesn’t survive a sub-make, a flag-ordering issue, or a silently-ignored unsupported flag is a real, recurring way hardening passes fail without anyone noticing – checking the compiled output is what closes that loop.

21.5.3 Closing the Bypass in the Measured-Boot Chain Itself

§21.4’s entire guarantee assumes new code only enters the kernel through a path IMA measures. `kexec` is a documented way around that assumption: `kexec_load` boots a completely different kernel image without a hardware reset, and without ever going back through firmware or bootloader measurement – the PCR values would simply never reflect it. The fix isn’t necessarily disabling `kexec` outright: Linux’s `kernel_lockdown` LSM [8] already gates `kexec_load` and permits `kexec_file_load` only for images signed by a trusted key [9] – the same signature-enforcement principle §21.4 already leans on for modules, applied to the one other path that can introduce an unmeasured kernel. Restricting `/dev/mem` and `/dev/kmem` (`CONFIG_STRICT_DEVMEM`) closes the more direct version of the same problem: unrestricted, either lets a sufficiently privileged process read or write physical memory directly, bypassing every mitigation in this section at once.

21.5.4 What Survives All of It: a Same-Privilege-Level Last Resort

Stacked, the mitigations above narrow the surviving attack down to a ROP/JOP or data-only exploit against already-loaded, already-measured code, with no accompanying info leak, evading the cross-checks in §21.6–§21.8. That is a substantially harder, more specific attacker than “any memory corruption,” but it is not zero – and for exactly that residual sliver, an active kernel-resident integrity monitor such as Linux Kernel Runtime Guard (LKRG) [19] can still add value, with one important condition on how it’s deployed.

LKRG snapshots kernel `.text` and critical structures (process credentials, security module state) and re-validates them on a mix of timer ticks and hooked kernel events. It is a real,

existing project, and its own honest limitation is a race: LKRG runs at the *same* privilege level as the exploit it’s trying to catch. An attacker fast enough to disable or patch out LKRG before its next check wins. This puts LKRG in a fundamentally different category from §21.4’s PCR-sealing – it is the one mechanism in this whole document explicitly *not* a same-privilege-level countermeasure; LKRG explicitly is one.

Report, Don’t Enforce

LKRG’s more aggressive response modes – killing the offending process, or panicking the box on detection – are a real, documented availability risk for a production server, and a fair reason to rule it out entirely if that’s the only mode on offer. It isn’t: run LKRG in logging/alert-only mode, and treat its findings the same way this document treats every other same-privilege-level signal – as one more contributor to the *content implausibility* bucket in §21.9, not as an autonomous enforcement action. The one place that gets to take a disruptive action stays OAN’s own deliberately-chosen fail-open/fail-closed relay policy (§8) – not a second, uncoordinated kill switch racing the attacker inside the kernel on its own terms. Deployed this way, LKRG must also be part of the measured baseline itself, loaded early enough that its own presence is expected at PCR-seal time, not an unmeasured surprise.

Layer	Removes	Cost / caveat
ASLR / KASLR + KPTI	Reliable exploitation without a separate info leak	Near zero; on by default on most current kernels
STRICT_KERNEL_RWX	Code-injection exploitation	May break legacy/out-of-tree modules; silent on ROP/JOP/data-only
Stack canaries	Stack-based control-flow hijack	Sub-1% overhead; scope-limited to stack overflows
kexec/devmem lock-down	Bypass of the measured-boot chain itself	Breaks unsigned-image kexec workflows unless images are properly signed
LKRG, report-only	Same-privilege-level detection of what survives every layer above	Real CPU overhead (low single digits, workload dependent); must run report-only in production, never enforcing

21.6 Cross-Checking Views That Should Never Disagree

PCR-sealing narrows the field; it doesn’t cover the memory-corruption gap above. The remaining mitigations in this section target exactly that gap – a live, correctly-keyed **kb-checker** whose *inputs* have been quietly poisoned – using observation paths this design already has, compared against each other rather than trusted individually.

A rootkit that hooks the syscall table to lie to **kb-checker** isn’t automatically clever enough to also hide from an eBPF hook watching the same fact from a different angle, and vice versa – maintaining two consistent lies across two independent code paths is strictly harder than maintaining one. Forwarding both views to a single comparison point (**kbd**, or OAN’s SBC over

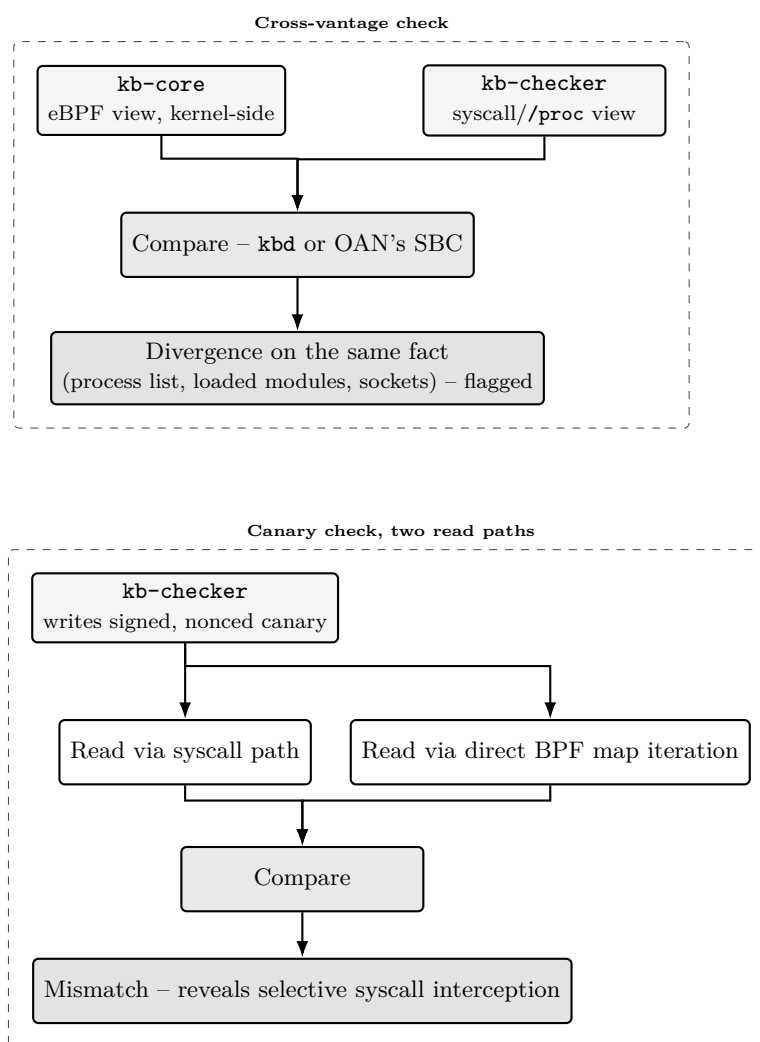


Figure 17: Two comparisons, not two new components. **kb-core** and **kb-checker** already observe overlapping facts through different code paths – the oldest rootkit-detection trick there is (raw vs. API view diffing) [30]. The canary variant targets the specific, most likely mechanism for lying to **kb-checker** without ever touching its binary: selective syscall interception.

the OOB link) costs nothing new to build; it’s a comparison, not a component.

The canary variant sharpens this further: have **kb-checker** periodically write a signed, nonced value into a BPF map, then read it back two ways – once through the normal syscall path, once through direct BPF map iteration. A rootkit intercepting one path but not the other reveals itself the moment the two answers disagree.

21.7 Timing, Not Just Content

Interception adds latency even when the forged output looks perfect – intercepting a syscall, computing a plausible lie, and returning it takes measurably longer than the kernel simply answering honestly. Have **kb-checker** include its own attestation-generation time in the signed payload, and have OAN’s SBC track the *distribution* of that latency over the heartbeat interval, not any single sample.

In Plain Terms

A single slow heartbeat is noise – a scheduler hiccup, a busy host. A sustained *shift in the shape* of that distribution is a side channel: evidence that something now sits between **kb-checker** and the kernel, independent of whether the reported content is plausible at all.

21.8 Cross-Checking the Clock Itself

OAN’s SBC has its own independent wall clock by construction – it is a physically separate computer (§5). Have **kb-checker**’s attestation carry a monotonic counter and an uptime figure, and have the SBC flag any inconsistency against the real elapsed time since the last good heartbeat. This catches stealth restarts, clock manipulation, or replay of an old, genuinely-valid attestation – cheap, and it uses nothing this design hasn’t already built.

21.9 Classify the Failure, Don’t Just Fence

Fencing alone answers “do something.” It doesn’t answer “what happened,” and a report that just says *OAN tripped* is barely more useful than silence itself. The mitigations above don’t need to share one fencing trigger – they need to feed one classification, attached to the recovery event and surfaced through FMS (§14).

Bucket	Trigger	What it means
Silence	No heartbeat arrives before timeout (§10)	Process-level failure – kb-checker killed, frozen, or starved. Could be a crash; could be malicious. CPM (§21.2) rules out KB’s own pipeline as the cause.
Cryptographic failure	Heartbeat arrives, TPM signature or PCR-policy check fails (§21.4)	High-confidence: measured-boot state changed unexpectedly. The closest thing to a direct rootkit/bootkit signature this design can produce.
Content implausibility	Signature valid, but canary, cross-vantage, or timing invariants fail (§21.6–§21.8)	kb-checker is alive and its key is intact, but something beneath it is lying to it – the confused-deputy case, specifically.

That classification is the difference between “OAN tripped” and “OAN tripped, category: cryptographic failure, consistent with an unauthorized kernel module load.” The second is something an operator can actually act on.

The same judgment – match the response to what’s actually needed – applies one layer down, at deployment time rather than incident time. **kbd.service** and **kb-checker.service** keep only a conservative baseline of systemd sandboxing directives [29], not the fuller capability-stripping and filesystem lockdown, because **kb-checker**’s own Tampering Containment Protocol (§12) genuinely needs root and **CAP_KILL**, **CAP_NET_ADMIN**, and **bpffs-delete** rights for its last-resort lockdown – a capability set gotten wrong there would fail silently until the one moment it matters most. **kbagents**, **kbopd**, and **kbopt** get the full lockdown, safe to apply precisely because none of them have anything resembling that need. The same principle either way:

hardening blind to what a component actually has to do isn't hardening, it's a future outage waiting for the day that need turns out to be real.

21.10 Letting Weak Signals Accumulate Across the Fleet

None of §21.6–§21.8 needs to be a hard trip-wire on its own – a single near-miss on one host might be noise. But `kb-aads` and FMS's cross-host correlation (§14, already a stated if underdesigned responsibility) can watch for the same *pattern* of near-misses showing up across multiple hosts around the same time. That is a much stronger signal of deliberate, coordinated compromise than any one host's isolated anomaly, and it costs nothing beyond forwarding signals that already exist to a layer that already exists.

Concretely: Host 3 logs one clock-drift near-miss (§21.8) at 02:14. Independently, Host 7 – a different Colony, connected to Host 3 only through FMS's signed summaries, nothing lower-level – logs a canary mismatch (§21.6) at 02:15. Neither event alone crosses that host's own fencing threshold; each looks, locally, like the scheduler-hiccup or transient-noise case §21.8 already warns against over-reacting to. Correlated by FMS across the fleet, the same ten-second window turns two individually-dismissible shrugs into one incident: a coordinated action against multiple hosts, timed together, that no single host's own local view could ever have surfaced on its own.

Patterns worth correlating, not just individually counting:

- **Classification-bucket clustering.** Several hosts landing in the same failure-classification bucket (§21.9) inside a short window, rather than the fleet's normal background rate of isolated, unrelated events.
- **Repeated interception fingerprints.** The same canary syscall path (§21.6) mismatching on more than one host – consistent with one exploit technique deployed fleet-wide, not independent, unrelated compromises.
- **PCR-seal failures tied to a shared update batch.** Multiple hosts that recently received the *same* patch or image failing PCR-sealed unseal around the same time points at a supply-chain question, not N separate local incidents.
- **Clock anomalies clustering in wall-clock time.** Hosts have independent oscillators (§21.8); truly unrelated drift does not line up in time. When several do line up, that alignment is itself the signal.

One caveat carries over directly from §14's Local Autonomy Principle: this correlation layer is an enhancement, never a dependency. Every host still evaluates and, if necessary, fences on its own local signal with zero reliance on FMS being reachable – fleet correlation adds context to an incident after the fact, it does not gate whether any single host protects itself.

21.11 What This Closes, and What Remains Open

Mitigation		Defends against	Residual gap
kb-checker CPM's floor	in	KB's own containment pipeline turning on its own watchdog	Nothing outside that pipeline
BPF map hardening (§21.2.1)		In-kernel or late-stage writes to CPM's own protected-list and containment maps	Writes from userspace where a map isn't frozen; freeze is one-way and only applied where nothing legitimate writes again
kb-core compiler hardening (§21.5.2)		Reliable memory-corruption exploitation of kb-core's own userspace daemon – the process managing CPM's maps and loading its eBPF programs	Same residual as the row below
Key moved into the host's TPM		Passive key extraction from host RAM	Active impersonation via the same signing API
PCR-sealed key usage (host TPM)		Impersonation by any attacker whose path to Ring 0 loaded new code	Memory-corruption-only exploits; physical TPM bus attacks; upstream firmware compromise
Memory-corruption narrowing (§21.5)		Reliable exploitation, code injection, stack hijack, and the measured-boot bypass, layered together	A ROP/JOP or data-only exploit with no info leak, evading every cross-check below
Cross-vantage & canary checks		Selective syscall interception that leaves kb-core's independent view untouched	A rootkit that fakes <i>both</i> views consistently
Timing & clock cross-checks		Interception latency; stealth restarts; attestation replay	Attacks with no measurable timing footprint
Failure classification		Turning a generic alarm into actionable evidence	Doesn't itself detect anything new
systemd hardening split		Excess privilege on the operator/agent services that don't need it	kbd/kb-checker keep broad privilege by documented necessity – hardening doesn't reach their own attack surface
Fleet-level correlation		Coordinated, multi-host compromise below any single host's hard threshold	A single, isolated, well-executed compromise on one host

Status

Most of this section is still design direction, at the same level of maturity as the open questions already carried elsewhere in this document – concrete enough to build from, not yet a contract. Four rows above are the exception, verified against the compiled output and the actual diff, not just described: `kb-checker`’s CPM protection, the BPF map hardening, `kb-core`’s compiler hardening, and the `systemd` hardening split are real and in the codebase today. §21.4 still needs a real design pass before it’s buildable: which PCR banks to bind to, the TPM2 policy-session mechanics (`tpm2-tools/tpm2-tss`, `TPM2_StartAuthSession` + `TPM2_PolicyPCR`), and where the re-sealing process lives operationally.

22 How OAN Compares to Existing Approaches

OAN’s shape – an independent management processor, a hardware root of trust, a physically separate network – is not invented from nothing. It follows patterns already proven at scale in adjacent fields, adapted to Kernel Borderlands’ specific job.

Approach	What it shares with OAN	Where OAN differs
Enterprise BMC / IPMI [22]	Independent management processor, dedicated network, works even if the main OS is down.	A BMC typically manages one server for remote administration; OAN’s primary purpose is security attestation and fencing, with fleet management (§14) built on top, not the other way around.
Hardware Security Module (HSM) [23]	Hardware-rooted key storage, tamper resistance.	An HSM protects keys for applications to use; OAN’s TPM protects an attestation and fencing decision, not application cryptography.
Academic out-of-band attestation research	Physically separate verifier, independent of the host’s own trust chain.	Most published designs stop at detection; OAN adds a deterministic, host-independent fencing action (§7.2) as a first-class part of the design, not a follow-on.

The honest framing is that OAN combines ideas each of these fields has already validated individually, applied together to one job: keeping Kernel Borderlands’ own trust anchor outside the reach of the exact compromise it exists to catch.

23 Frequently Asked Questions

Can an attacker just unplug OAN?

Physically, yes – and that is a deliberate, visible action rather than a silent software bypass. Because `kb-checker` expects a heartbeat response on a fixed schedule (§10), losing the OOB link at all is itself the failure condition the fail-closed policy is built to catch: no response, past timeout, triggers fencing just as surely as an invalid response would.

What stops someone from swapping OAN’s SBC for a fake one that always reports “healthy”?

The TPM. A forged SBC has no path to the sealed measurements the real SBC’s measured boot produced, so it cannot produce a verification the TPM accepts (§8). This is exactly the property a software-only check could not offer.

Does OAN need to trust the network it runs on?

The OOB Trust Plane, no – it is physically isolated from any production network by design (§6). The Fabric Management Plane runs on ordinary networking, but it never carries anything more sensitive than a signed summary that a compromised network cannot forge (§14).

What happens during a legitimate reboot or maintenance window?

An operator-initiated maintenance mode is expected to suppress fencing for a bounded window, the same way any watchdog system needs a sanctioned way to say “this outage is expected.” The heartbeat and timeout mechanics themselves (§10) are unchanged – only whether the STM32 is allowed to act on a missed one.

Why five hosts per chassis, and not, say, ten or twenty?

Blast radius, commodity part sizes, and the SBC’s USB/GPIO budget all independently point at the same number – see §13’s “Why Cap at Five Hosts” for the full reasoning.

Is OAN required to use Kernel Borderlands at all?

No. Every host keeps full protection from `kb-core`, `kb-control-plane`, and `kb-checker` with no OAN present at all. OAN adds a hardware trust anchor on top of an already-complete software stack; it is a hardening layer, not a dependency.

24 Positioning: What OAN Adds to Kernel Borderlands

OAN extends Kernel Borderlands beyond a software-only runtime defense framework by introducing a genuinely independent, hardware-assisted trust layer. It gives the platform a verification domain that can monitor host integrity, validate runtime attestation, coordinate recovery, and perform last-resort fencing *without ever depending on the operating system it is protecting* – closing the one gap a same-host watchdog cannot structurally close on its own, and giving the platform a foundation that scales from a single protected desk (§5) to a coordinated fleet (§13 – §14) on the same trust model throughout.

25 From Design to Deployment: The Build Roadmap

Everything in this document describes one continuous architecture, but it is built in three phases, each one a strict superset of the hardware and guarantees the phase before it already established.

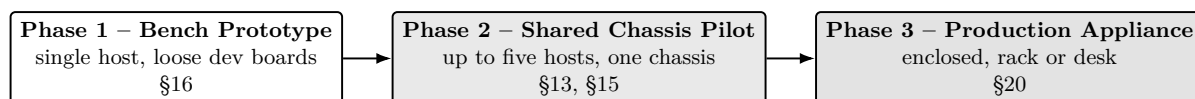


Figure 18: Each phase reuses the same three-chip architecture (§7), the same TPM-rooted trust chain (§8), and the same recovery ladder (§12). What changes across phases is packaging and host count, never the trust model.

Phase	Goal	Exit condition
1. Bench Prototype	Prove the attestation heartbeat and fencing loop end to end on one host, using loose commodity dev boards.	A full heartbeat cycle (§10) succeeds repeatedly, and a deliberately broken link triggers the configured fencing policy (§27).
2. Shared Chassis Pilot	Validate the shared-hardware economics (§16) and the fleet layer (§14) on a real, multi-host chassis.	Five hosts registered with FMS, organized into Families, reporting signed summaries continuously.
3. Production Appliance	Package the validated design into an enclosed unit (§18) suitable for a rack or a shared desk, not a breadboard.	Front/rear panel design finalized; internal layout addresses cooling and power regulation at sustained load.

In Plain Terms

No phase requires re-deciding anything about *trust* – only about *packaging and scale*. The TPM still seals the same two things (§8), the STM32 still owns the relay alone (§7.2), and Stage 5 is still the last resort (§12) whether Phase 1’s prototype is sitting on a breadboard or Phase 3’s appliance is bolted into a rack.

26 Glossary

ASLR/KASLR (Kernel) Address Space Layout Randomization – randomizes where code and data land in memory on every boot, so a memory-corruption bug that doesn’t know where anything is usually just crashes rather than achieving reliable code execution [10].

Attestation A signed statement from one component to another proving “I am in the state I claim to be in,” backed by cryptography rather than trust alone.

BMC Baseboard Management Controller – the enterprise-server equivalent of OAN’s role: a small, independent management processor that keeps working even if the main OS is down [22].

Confused Deputy

A program that is tricked into misusing its own legitimate authority on an attacker’s behalf [2] – the framing §21 uses for a **kb-checker** that is still running but has been fed false data.

eBPF Extended Berkeley Packet Filter [26, 27] – the technology **kb-core** uses to observe kernel activity at Ring 0 without modifying the kernel itself.

Fencing	Physically isolating a compromised machine – cutting its power, its reset line, or its network access – so it can no longer act, whether or not its software cooperates.
Heartbeat	A signal sent at a fixed interval to prove a component is still alive and functioning correctly.
IMA	Integrity Measurement Architecture [6, 7] – standard Linux, no new hardware – extends a PCR every time a kernel module loads.
IWDG	Independent Watchdog – a hardware timer peripheral on the STM32 that resets or triggers an action if not “fed” in time.
Kernel Ring 0	The most privileged execution level on a CPU, where the operating-system kernel itself runs.
KPTI	Kernel Page-Table Isolation – closes Meltdown-class speculative-execution reads of kernel memory from userspace [17], which indirectly closes one of the easiest ways to defeat KASLR before triggering a bug.
LKRG	Linux Kernel Runtime Guard [19] – an active, same-privilege-level kernel integrity monitor; see §21.5.4 for why it belongs in report-only mode, not enforcement mode.
Measured Boot	Firmware, bootloader, kernel, and initrd each hash-measure the next stage before executing it, extending the result into TPM PCRs – the chain §21.4’s PCR-sealing depends on.
OOB	Out-of-Band – a communication channel that is physically or logically separate from a system’s normal, in-band data path.
PCR	Platform Configuration Register [4, 5] – a TPM register that is only extendable, never directly writable, forming a one-way hash chain rooted at boot.
Root of Trust	The one component in a system that everything else’s trust is ultimately built on – if it is compromised, no higher-level check can be relied on.
Rootkit	Malware that hides its own presence, typically by subverting the kernel or a privileged process it runs inside [30].
ROP/JOP	Return-Oriented / Jump-Oriented Programming – exploit techniques that reuse already-legitimate, already-executable code instead of injecting new code, so W ^X [13] is silent on them [15, 16].
TPM	Trusted Platform Module – a dedicated hardware chip for secure key storage, measured boot, and integrity attestation.
Watchdog	A component whose job is to detect when something else has failed and to take a predefined recovery action.

W[^]X Write XOR Execute – a memory page is either writable or executable, never both [13], blocking the classic “inject shellcode, jump to it” technique but not ROP/JOP.

27 Build Checklist

A practical, ordered checklist for a team building a first shared chassis (§13), grouped by the same stages as the deployment walkthrough (§15).

27.1 Before Ordering Parts

- ☐ Confirm the host machine has a free USB or serial port, and a TPM header/SPI bus if the discrete TPM will sit on the host side of any wiring.
- ☐ Decide fail-open vs. fail-closed per host class (§8) before wiring, not after.
- ☐ Choose reference-cost or budget-cost parts (§16 vs. §17) – the discrete TPM and the STM32 split are never optional, at either budget.
- ☐ Decide how many hosts this first chassis will serve (1–5) – the shared parts are bought once regardless, so under-provisioning per-host parts is the only real risk of ordering too little.

27.2 Assembling the Shared Chassis

- ☐ Flash the SBC’s microSD with the base OS and the attestation engine.
- ☐ Wire the TPM module to the SBC and verify measured boot produces a stable, repeatable sealed measurement.
- ☐ Flash the ESP32 and confirm it can reach the dedicated OOB switch, and *cannot* reach any production network segment.
- ☐ Connect the optional display and confirm it reflects SBC/STM32 state (read-only, §8).

27.3 Wiring the First Host

- ☐ Flash the STM32 channel with the deterministic watchdog firmware; confirm its IWDG fires correctly on a bench test *before* wiring it to a real host’s relay.
- ☐ Wire the relay to the host’s power/reset header – verify the NO/NC contacts match the intended fail-open/fail-closed policy for this host.
- ☐ Connect the dedicated OOB dongle, distinct from the host’s main NIC.
- ☐ Install and start **kb-checker** on the host, and confirm a full heartbeat cycle (§10, steps 1–4) succeeds repeatedly before trusting the link.
- ☐ Deliberately break the link (unplug the dongle) and confirm the configured failure policy actually fires within the expected timeout – an untested failure path is not a working one.

27.4 Scaling the Chassis

- ☐ Repeat “Wiring the First Host” for each additional host, up to the five-host cap (§13).

- Register each new Node with FMS and assign it to a Family (§14).
- At the cap, provision a second independent chassis rather than extending this one (§13’s blast-radius rationale).

27.5 Hardening Against Boot-Time and Rootkit Compromise (Optional, §21)

- Confirm the *host’s own* TPM 2.0 is present and enabled in firmware (Intel PTT / AMD fTPM, or discrete) – this is separate from OAN’s own TPM and does the job described in §21.4, not the relay job in §8.
- Seal **kb-checker**’s OOB signing key to the host’s own PCR values via `TPM2_PolicyPCR`; verify the unseal succeeds on a clean boot and fails after a simulated PCR extend, using the PCR 16 sandbox walkthrough (§21.4.3) before touching PCRs 0–10 on a production host.
- Verify **kb-core**’s userspace binaries actually carry the compiler-hardening flags against the compiled output, not just the Makefile: `readelf` for PIE and full RELRO, `nm` for the stack-protector symbol (§21.5.2).
- Enable Linux’s `kernel_lockdown` LSM; confirm `kexec_load` is blocked and `kexec_file_load` only accepts images signed by a trusted key. Restrict `/dev/mem` and `/dev/kmem` (`CONFIG_STRICT_DEVMEM`).
- Decide whether to deploy LKRG for this host class. If yes, run it in logging/alert-only mode (§21.5.4) – never its process-kill/panic modes – and fold its own presence into the measured baseline *before* sealing PCRs, so it is expected at PCR-seal time rather than flagged as drift.
- Confirm the systemd hardening split from `boot_sequence_spec.md` is actually applied per unit – conservative baseline on `kbd.service/kb-checker.service`, full lockdown on `kbagents.service/kbopd.service/kbopt.service`.

28 Summary Cheat-Sheet

A one-page recap of the facts a reader is most likely to need again later, without re-reading the full document.

What	Answer
What OAN is	A physically separate hardware appliance that attests to and can fence a Kernel Borderlands host, independent of that host’s own kernel.
What it talks to on the host	Only kb-checker , over a dedicated out-of-band link (§5).
Its three processors	SBC (brain), STM32 (spinal reflex), ESP32 (mouth) – §7.
Its root of trust	A discrete TPM 2.0, sealing both OAN’s own state and the host’s reported state (§8).
Its failure policy	Configurable; defaults to fail-closed (§8).
Where it sits in recovery	Stage 5 of 5, strictly below kb-checker ’s existing software chain (§12).

What	Answer
Reference deployment model	One shared chassis per up to five hosts (§13).
Fleet hierarchy	Fabric → Colony (one chassis) → Family (purpose grouping) → Node (one host) – §14.
Cost, one host	\$106–195 / INR 9,220–16,965 reference; \$54–76 / INR 4,698–6,612 budget (§16, §17).
Cost, five hosts (one chassis)	\$234–383 / INR 20,360–33,320 reference; \$114–168 / INR 9,918–14,616 budget.
What is never cut, at any budget	The discrete TPM and the STM32 split (§17).
What FMS can and cannot weaken	Coordinates the fleet; never sits in any single host’s runtime security path (§14.1).

29 Appendix: Document Map

This document is a plain-language synthesis. The full engineering detail behind each section lives in the Kernel Borderlands documentation tree:

Section here	Source documentation
Problem, philosophy, architecture, hardware, heartbeat, recovery ladder	<code>docs/features/hardware-ext/oan-hardware-appliance.md</code>
Fleet scaling, Fabric → Colony → Family → Node, FMS responsibilities	<code>docs/features/hardware-ext/oan-fms.md</code>
Budget build, cost-reduction levers	<code>docs/features/hardware-ext/oan-cheap.md</code>
Boot/rootkit compromise resistance, PCR-sealing, this section’s markdown twin	<code>docs/features/hardware-ext/oan-rootkit-resistance.md</code>
<code>kb-checker</code> ’s existing software recovery chain	<code>kb-checker/README.md</code>
Critical Process Module, “fail closed toward protection”	<code>docs/features/CPM.md</code>
Critical Workload Protection	<code>docs/features/CWP.md</code>
Socket topology, boot ordering	<code>docs/architecture/boot_sequence_spec.md</code>

30 The Complete Picture: Full System Diagram

One last, comprehensive view, folding every diagram in this document into one picture. It is turned 90° on the page – unlike every other, upright figure in this document – purely because its content is wide: rotating it is what lets every layer, from the fleet down to a single host’s own internal stack, stay at a readable size instead of being shrunk to fit a portrait column.

The complete picture, top to bottom: the Fleet layer (FMS coordinating multiple Colonies), one Colony’s shared OAN chassis, its per-host STM32/relay channels, the protected hosts themselves, and one host’s own existing Kernel Borderlands software stack – all five subsystems (**kb-core**, **kb-control-plane**, **kb-op**, **kb-aads**, **kb-checker**) – underneath it all.

Figure 19 (next page) packs a lot into one rotated view. Read top to bottom, here is what every labeled box in it is:

Component	Role in the diagram
Fabric & FMS	Coordinates every Colony in the fleet (§14) – non-security-critical, on its own Fabric Management Plane, never in a single host’s runtime security path.
Other Colonies	The rest of the fleet – each an independent, self-contained shared chassis exactly like the one drawn in full.
SBC (shared)	The chassis’s attestation/policy engine – “the brain” (§7). Runs the software that talks TPM and drives the fleet link.
ESP32 (shared)	“The mouth” – management/comms only, kept out of the trust-critical path due to its larger, network-facing attack surface.
TPM (shared)	OAN’s own root of trust (§8) – seals OAN’s own integrity and verifies what kb-checker reports. Distinct from any host’s own TPM (§21.4).
Display (shared)	Optional, read-only reflection of SBC/STM32 state – never a control surface.
STM32 ch. 1–5	“The spinal reflex” – one deterministic safety-controller channel per host, each running the heartbeat/fencing logic independent of the SBC’s own Linux failure domain.
Relay	The physical fencing actuator each STM32 channel drives – cuts a host’s power or reset line on a fencing decision.
Host 1–5	The protected machines themselves, each wired to its own STM32 channel and carrying its own dedicated OOB link (Figure 3).
kb-core	Ring-0 eBPF sensors on the host – the deepest existing KB vantage point, and what the OOB Trust Plane ultimately corroborates.
kb-control-plane (kbd)	Ingests kb-core ’s telemetry over kbd.sock , scores it, and pushes containment/control back down over the separate kbctl.sock .
kb-checker	The existing software watchdog – watches the rest of the stack independently, and is the one component that also feeds the OOB Trust Plane up to OAN’s SBC (Figure 2).
kb-op	Operator interfaces – kb-tui , kb-dashboard , kbctl , kb-mcp – talking to kbd over gRPC/WebSockets.
kb-aads	The Python agent swarm, driven by the same kbd API as kb-op .

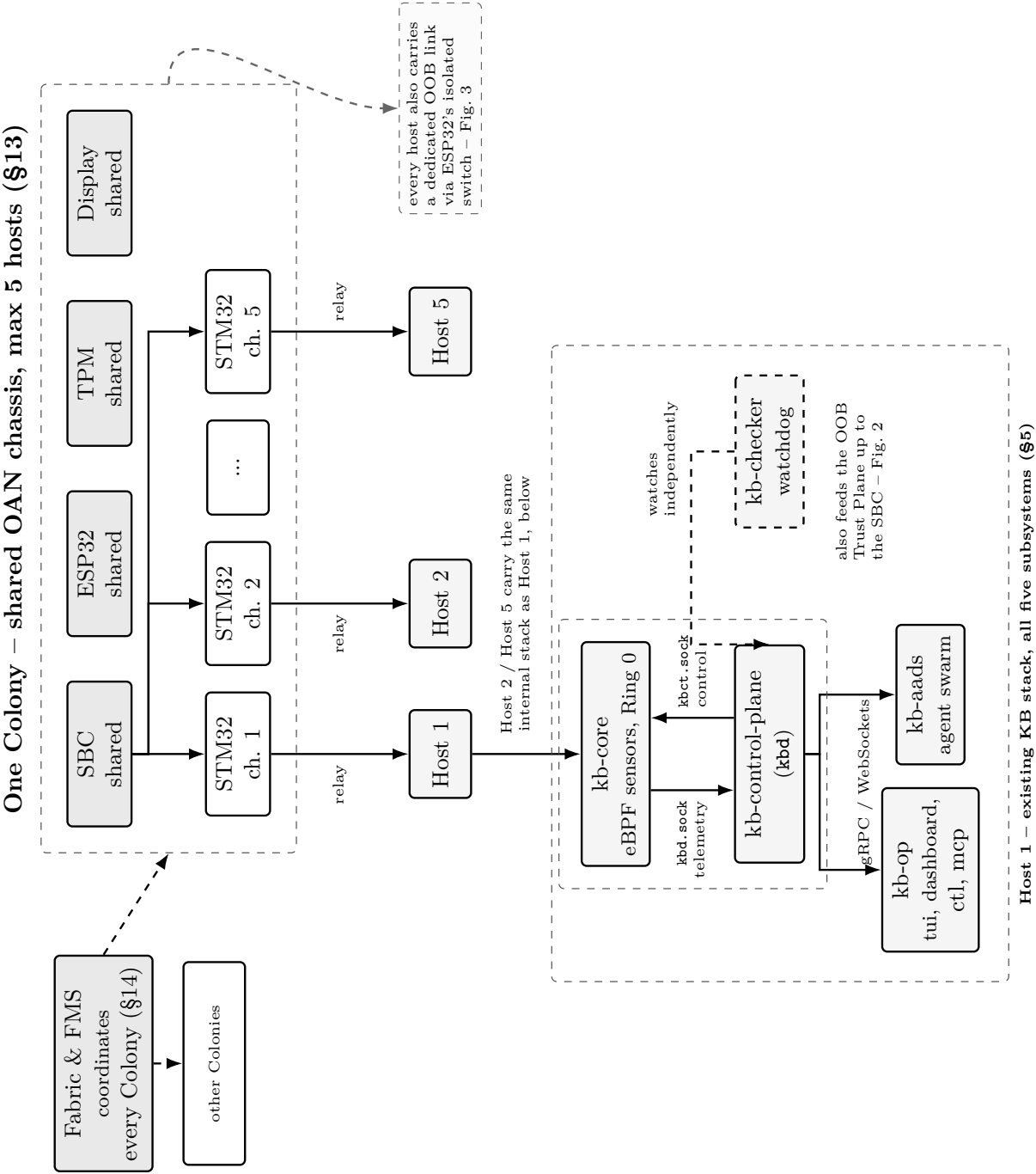


Figure 19: The complete picture, top to bottom.

31 References

- [1] Kernel Borderlands project repository. <https://github.com/pardhuvarmax/kernel-borderlands/tree/main>.
- [2] N. Hardy, “The Confused Deputy: (or why capabilities might have been invented),” *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.
- [3] M. Bishop and M. Dilger, “Checking for Race Conditions in File Accesses,” *Computing Systems*, vol. 9, no. 2, pp. 131–152, 1996.
- [4] Trusted Computing Group, *Trusted Platform Module Library Specification, Family “2.0”*. <https://trustedcomputinggroup.org>.
- [5] Trusted Computing Group, *PC Client Platform Firmware Profile Specification*. <https://trustedcomputinggroup.org>.
- [6] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, “Design and Implementation of a TCG-based Integrity Measurement Architecture,” in *Proc. 13th USENIX Security Symposium*, 2004.
- [7] *Integrity Measurement Architecture (IMA)*, The Linux Kernel Documentation. <https://www.kernel.org/doc/>.
- [8] *Kernel Lockdown LSM*, The Linux Kernel Documentation. <https://www.kernel.org/doc/>.
- [9] *Kernel Module Signing Facility*, The Linux Kernel Documentation. <https://www.kernel.org/doc/>.
- [10] PaX Team, *PaX Address Space Layout Randomization (ASLR)*, design document, 2003.
- [11] C. Cowan, C. Pu, D. Maier, H. Hinton, J. Walpole, P. Bakke, S. Beattie, A. Grier, P. Wagle, and Q. Zhang, “StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks,” in *Proc. 7th USENIX Security Symposium*, 1998.
- [12] Intel Corporation, *Control-flow Enforcement Technology Specification*.
- [13] T. de Raadt, “Exploit Mitigation Techniques,” OpenBSD Project (W^X design), 2005.
- [14] Solar Designer, “Getting around non-executable stack (and fix),” Bugtraq mailing list, 1997.
- [15] H. Shacham, “The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86),” in *Proc. ACM Conference on Computer and Communications Security (CCS)*, 2007.
- [16] T. Bletsch, X. Jiang, V. Freeh, and Z. Liang, “Jump-Oriented Programming: A New Class of Code-Reuse Attack,” in *Proc. 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011.
- [17] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, “Meltdown: Reading Kernel Memory from User Space,” in *Proc. 27th USENIX Security Symposium*, 2018.

- [18] D. Gruss, M. Lipp, M. Schwarz, R. Fellner, C. Maurice, and S. Mangard, “KASLR is Dead: Long Live KASLR,” in *Proc. International Symposium on Engineering Secure Software and Systems (ESSoS)*, 2017.
- [19] Openwall Project, *Linux Kernel Runtime Guard (LKRG)*. <https://github.com/openwall/lkrg>.
- [20] Intel Corporation, *Platform Trust Technology (PTT)*, part of Intel Trusted Execution Technology documentation.
- [21] AMD, *AMD Secure Technology — Firmware TPM (fTPM)* documentation.
- [22] Distributed Management Task Force / Intel, *Intelligent Platform Management Interface (IPMI) Specification*. <https://www.dmtf.org>.
- [23] National Institute of Standards and Technology, *FIPS 140-3: Security Requirements for Cryptographic Modules*, 2019.
- [24] National Institute of Standards and Technology, *NIST SP 800-155: BIOS Integrity Measurement Guidelines* (draft).
- [25] UEFI Forum, *Unified Extensible Firmware Interface (UEFI) Specification* (Secure Boot). <https://uefi.org>.
- [26] S. McCanne and V. Jacobson, “The BSD Packet Filter: A New Architecture for User-level Packet Capture,” in *Proc. USENIX Winter Conference*, 1993.
- [27] *BPF and XDP Reference Guide*, The Linux Kernel Documentation. <https://www.kernel.org/doc/>.
- [28] tpm2-software Community, *tpm2-tools* and *tpm2-tss*. <https://github.com/tpm2-software>.
- [29] *systemd.exec(5)* — sandboxing directives (NoNewPrivileges=, ProtectSystem=, CapabilityBoundingSet=). <https://systemd.io>.
- [30] G. Hoglund and J. Butler, *Rootkits: Subverting the Windows Kernel*. Addison-Wesley, 2005.
- [31] Stefan Berger et al., *swtpm — TPM Emulator for QEMU/libvirt*. <https://github.com/stefanberger/swtpm>.
- [32] Trusted Computing Group, *TCG PC Client Platform TPM Profile (PTP) Specification* — PCR index assignment (including the reserved/resettable “debug” PCRs used in §21.4.3).

KERNEL BORDERLANDS

Thank You

for reading through the Out-of-Band Attestation Node design

TEAM KERNEL BORDERLANDS

<https://github.com/pardhuvarmax/kernel-borderlands>